

A Project Report On

**Empirical Evaluation of Decentralized
Federated Learning: A Gossip-Based
Approach on Cloud Infrastructure**

Submitted in partial fulfilment of the requirements
for the award of the degree of

Master of Technology
In
Information Technology

By
Tirthoraj Bhattacharya (MSE2024008)
(4th Semester)

Under the Supervision of
Dr. Anshu S. Anand



Department of Information Technology
Indian Institute of Information Technology, Allahabad
Devghat, Jhalwa, Prayagraj-211015 (UP) INDIA

May, 2026



Indian Institute of Information Technology, Allahabad
Devghat, Jhalwa, Prayagraj-211015 (UP) INDIA

CERTIFICATE

This is to certify that the project entitled “**Empirical Evaluation of Decentralized Federated Learning: A Gossip-Based Approach on Cloud Infrastructure**” submitted by Mr. **Tirthoraj Bhattacharya (MSE2024008)** to Indian Institute of Information Technology Allahabad, Prayagraj, Uttar Pradesh, in partial fulfilment for the award of Degree of Master of Technology in Information Technology is a bonafide record of the project work carried out by him under my supervision during the academic year 2025–26.

Dr. Anshu S. Anand
Dept. of Information Technology
IIIT Allahabad



Indian Institute of Information Technology, Allahabad
Devghat, Jhalwa, Prayagraj-211015 (UP) INDIA

ACKNOWLEDGEMENT

I express my sincere gratitude to all those who have supported me throughout this research. I am deeply thankful to my supervisor, **Dr. Anshu S. Anand**, for his invaluable guidance, encouragement, and constructive feedback at every stage of this work. His mentorship was instrumental in shaping the research direction and ensuring academic rigour throughout.

I also extend my appreciation to the faculty and staff of the Department of Information Technology at IIIT Allahabad for providing a supportive environment for academic exploration. I am grateful to my peers and colleagues for their discussions and encouragement, which enriched both my understanding of the subject and the quality of this work.

Finally, I acknowledge the researchers behind the foundational works in federated learning and decentralized gossip protocols — particularly McMahan et al. (FedAvg), Lian et al. (D-PSGD), and Lalitha et al. (Gossip FL) — whose contributions provided the theoretical basis upon which this empirical investigation was built.

Tirthoraj Bhattacharya
MSE2024008

ABSTRACT

Federated Learning (FL) allows multiple devices to train a shared model without sharing their raw data. The standard approach, FedAvg, depends on a central server to coordinate every training round. If that server goes down, the entire system halts — a Single Point of Failure (SPOF) that is widely discussed in the literature but never measured on real distributed infrastructure.

This thesis compares centralised FedAvg and decentralised gossip FL, both deployed on 8 real AWS EC2 instances using CIFAR-10 and a standard CNN. Six experiments cover normal training under IID and Non-IID data, plus live failure injection in both architectures.

The main findings are: decentralised gossip reaches within 4.2 percentage points of FedAvg accuracy under both data conditions. When the server is deliberately killed, all 7 clients stop permanently at round 9 and never recover. When one node is killed in the decentralised setup, the remaining 7 nodes complete all 50 rounds without halting. Additionally, a previously undocumented cascading failure behaviour was observed — a failed node’s TCP timeout inflates its neighbours’ round time, which eventually cuts them off from the rest of the ring entirely.

The results show that decentralised FL eliminates a critical failure mode at a modest accuracy cost, and reveal real-world system behaviours that theory alone does not predict.

Keywords: Federated Learning, Gossip Protocol, FedAvg, Fault Tolerance, SPOF, AWS EC2, CIFAR-10, Non-IID

CONTENTS

LIST OF ABBREVIATIONS	vi
1 Introduction	1
1.1 Background	1
1.2 Contributions	2
2 Literature Review	4
2.1 Federated Learning Foundations	4
2.2 Non-IID Data in Federated Learning	4
2.3 Decentralised Federated Learning	5
2.4 The Gap This Thesis Fills	6
3 Problem Formulation	7
3.1 Problem Statement	7
3.2 Research Questions	7
3.3 Design Decisions	8
3.4 Scope and Limitations	9
4 System Design and Architecture	10
4.1 Centralised FedAvg Architecture	10
4.2 Decentralised Gossip FL Architecture	11
4.3 Design Comparison	11
4.4 Infrastructure	12
4.5 Communication Protocol Detail	13
5 Experimental Methodology	14
5.1 Infrastructure	14
5.2 Model and Dataset	14
5.3 Data Distribution	15
5.4 Hyperparameters	15
5.5 Pilot Study	15
5.6 The Six Experiments	16
6 Results and Discussion	17
6.1 Pilot Study: 2-VM VMware Results	17
6.2 Centralised FedAvg: Experiments 1 and 2	18
6.2.1 IID Results (Experiment 1)	18
6.2.2 Non-IID Results (Experiment 2)	19

6.3	Decentralised Gossip FL: Experiments 3 and 4	23
6.3.1	IID Results (Experiment 3)	23
6.3.2	Non-IID Results (Experiment 4)	24
6.4	Architecture Comparison: Experiments 1–4	27
6.5	Fault Injection: Experiments 5-A and 5-B	28
6.5.1	Experiment 5-B: SPOF in Centralised FL	28
6.5.2	Experiment 5-A: Fault Tolerance in Decentralised FL	29
6.5.3	The Cascading Timing-Drift Effect	31
6.5.4	Direct Comparison of Fault Outcomes	31
7	Conclusion	33
7.1	Summary of Work	33
7.2	Key Findings	33
7.3	Limitations	34
7.4	Future Work	35
	REFERENCES	35

LIST OF FIGURES

1.1	Standard Federated Learning Workflow	1
4.1	Centralised FedAvg — Star Topology (1 Server + 7 Clients)	11
4.2	Decentralised Gossip FL — Ring Topology (8 Peer Nodes)	12
4.3	Communication protocol for one training round.	13
6.1	2-VM pilot — IID results. Left: decentralised (both VMs complete 30/30 rounds, 36.68%). Right: centralised (VM2 disconnects at Round 17 due to server broadcast failure — the motivating SPOF observation for Experiment 5-B).	18
6.2	2-VM pilot — Non-IID $\alpha=0.1$ results. Identical final accuracy (28.22%) in both architectures. Client drift (DOWN trend for Node 1/Client 1) is identical in both, confirming implementation correctness.	18
6.3	Centralized FedAvg — IID experiment, early training (Round 2). All 7 clients upload to the server; server broadcasts the aggregated model.	19
6.4	Centralized FedAvg — IID experiment, late training (Round 48). Convergence is stable with all clients reaching 60%+ local accuracy.	19
6.5	Centralized FedAvg — Non-IID experiment, early training (Round 4). Large per-round FedAvg boosts visible as clients' local accuracy is far below the aggregated global accuracy.	21
6.6	Centralized FedAvg — Non-IID experiment, late training (Round 48). Aggregated accuracy stabilises around 56–58% despite local accuracy remaining well below that range.	21
6.7	Global accuracy vs. round for Experiments 1 and 2 (Centralised FedAvg). IID peaks at 61.69% (Round 28); Non-IID peaks at 57.60% (Round 26). The gap stabilises at 3.4–5.1 pp from Round 12 onward. Filled markers indicate peak accuracy points.	22
6.8	Decentralised gossip — IID experiment, Round 3. Each node pushes its model to both ring neighbours and blends three models (own + left + right).	23
6.9	Decentralised gossip — IID experiment, Round 46. Nodes converge within a tight accuracy band with no central coordinator.	24
6.10	Decentralised gossip — Non-IID experiment, Round 8. PUSH retry attempts visible as some nodes take longer to synchronise with neighbours whose training speed differs.	25

6.11	Decentralised gossip — Non-IID experiment, Round 47. Accuracy spread across nodes is wider than under IID, reflecting the limited complementary knowledge in a 2-neighbour ring under heterogeneous data.	25
6.12	Post-blend accuracy vs. round for Experiments 3 and 4 (Decentralised Gossip, Node 0). IID converges to $\sim 57\%$ and holds steadily from Round 15. Non-IID plateaus at $\sim 51\text{--}52\%$ from Round 15, never crossing 53% sustainably. The gap between IID and Non-IID (~ 7 pp) is wider than in centralised FL (~ 3.5 pp), confirming gossip's greater sensitivity to data heterogeneity.	26
6.13	Per-round accuracy of Nodes 2 and 4 (direct neighbours of failed Node 3) in Experiment 5-A. Phase 1 (R1–10): normal operation. Phase 2 (R11–37/41): single surviving neighbour, accuracy held but comm time inflated. Phase 3 (R38/41–50): complete isolation, accuracy frozen at local training output ($\sim 43\text{--}44\%$).	27
6.14	Centralised SPOF — Round 5, normal operation just before the server failure at Round 10.	29
6.15	AWS EC2 infrastructure showing the 4 terminal windows from the centralised SPOF experiment. After the server exits, all client terminals show repeated connection refused errors.	29
6.16	Decentralised fault tolerance — Node 3's neighbours (Nodes 2 and 4) showing PUSH retry attempts after Node 3 exits. Both continue training with their surviving neighbour.	30
6.17	Decentralised fault tolerance — Round 46. All surviving nodes continue making progress, with Nodes 2 and 4 showing reduced accuracy due to limited blending after Node 3's exit.	30

LIST OF TABLES

2.1	Comparison of decentralised FL works	6
4.1	Architecture Comparison	12
5.1	Hyperparameters used across all experiments	15
5.2	Summary of all six experiments	16
6.1	Local vs. aggregated accuracy for Client 1 under Non-IID (Exp 2). FedAvg boost = aggregated – local.	20
6.2	Convergence milestones — Centralised FedAvg	22
6.3	Per-node final accuracy — Decentralised Gossip (Round 50)	26
6.4	Architecture comparison — normal operation results	27
6.5	Communication time inflation by ring distance from failed Node 3	31
6.6	Fault experiment outcomes: decentralised vs centralised	32

LIST OF ABBREVIATIONS

AWS	Amazon Web Services
CNN	Convolutional Neural Network
CIFAR	Canadian Institute For Advanced Research (dataset)
CPU	Central Processing Unit
EC2	Elastic Compute Cloud
FL	Federated Learning
FedAvg	Federated Averaging
GDPR	General Data Protection Regulation
IID	Independent and Identically Distributed
ML	Machine Learning
Non-IID	Not Independent and Identically Distributed
P2P	Peer-to-Peer
RAM	Random Access Memory
SGD	Stochastic Gradient Descent
SPOF	Single Point of Failure
TCP	Transmission Control Protocol
VM	Virtual Machine
VPC	Virtual Private Cloud

Chapter 1

Introduction

1.1 Background

Federated Learning (FL), introduced by McMahan et al. [1], allows multiple devices to train a shared machine learning model without sharing their raw data. Each device trains locally on its own data and only sends model weight updates to a central server. The server combines all received updates into a new global model and sends it back to the devices. This process repeats over multiple rounds until the model converges. Since the raw data never leaves the device, FL is naturally suited for applications where data privacy is important — healthcare, finance, and edge computing being common examples.

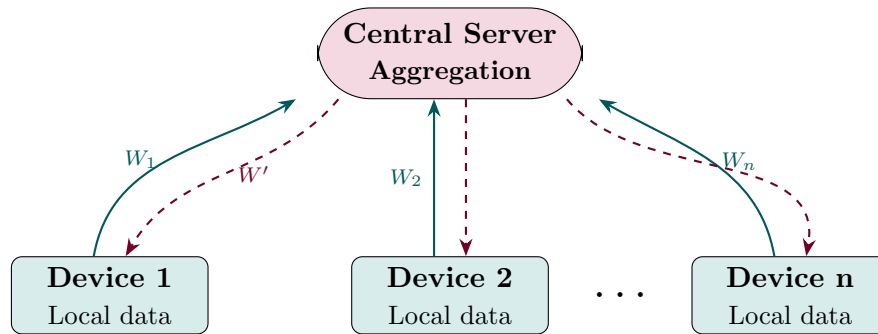


Figure 1.1: Standard Federated Learning Workflow

The standard FL algorithm that implements this is Federated Averaging (FedAvg). It is simple, well-understood, and works well under controlled conditions. However, it has one structural problem that becomes critical in real-world deployments.

In FedAvg, the central server is not just a coordinator — it is the only entity that can aggregate models and push the training forward. Every client must upload to it, wait for it, and download from it every single round. This creates a hard dependency: **if the server goes down at any point during training, every client stops immediately**. There is no alternative path, no backup, and no way to

resume without restarting the server manually. In distributed systems, this kind of dependency is called a **Single Point of Failure (SPOF)**.

In a research setting, this is manageable — if the server crashes, you restart and try again. But in real-world FL deployments — hospitals sharing patient data across branches, factories running FL across machines on a shop floor, or telecom networks training models across hundreds of base stations — server downtime can mean hours of lost training, corrupted experiment state, and no guarantee of recovery. The larger the deployment, the more damaging an unexpected server failure becomes.

The research community recognises this problem. Most major FL survey papers identify the SPOF vulnerability of centralised architectures as one of the key barriers to real-world adoption. The proposed solution is **decentralised FL** — where nodes communicate directly with each other in a peer-to-peer network instead of routing everything through a central server. If one node fails in such a setup, the others can continue exchanging models and training without interruption. No single node's failure can bring the entire system down.

This sounds like a clean solution, and theoretically it is. Convergence proofs exist showing that decentralised gossip FL can match centralised FL performance under ideal conditions. But here is the gap: **none of this has been tested on real distributed infrastructure with live failure injection**. Every fault tolerance claim in the decentralised FL literature is backed by either a mathematical proof or a simulation. What actually happens when a server or node is killed mid-experiment on real machines has never been directly measured and reported.

This thesis fills that gap. It implements both a centralised FedAvg system and a decentralised gossip FL system from scratch, deploys them on 8 real AWS EC2 cloud instances, and runs six controlled experiments — including deliberately killing the server in one experiment and a ring node in another — to observe and measure the real outcome in both cases.

1.2 Contributions

The specific contributions of this thesis are:

1. A live demonstration of the SPOF problem in centralised FL on real cloud infrastructure, with timestamped log evidence from actual AWS EC2 instances — not a simulation.
2. The first measured accuracy-fault tolerance tradeoff between synchronous gossip FL and FedAvg under identical hardware and data conditions.
3. Discovery of a cascading failure behaviour in synchronous ring gossip caused by TCP timeout design — an emergent real-world behaviour not described in any existing FL paper.

-
4. A Non-IID sensitivity comparison between both architectures, with an explanation of why gossip FL degrades more under heterogeneous data.
 5. A complete open implementation of both architectures on 8 real AWS EC2 instances with full experiment logs.

Chapter 2

Literature Review

This chapter surveys the existing work that directly leads to the research question this thesis addresses. It covers the foundations of federated learning, the challenges of non-IID data, the theoretical basis for decentralised FL, and ends with the specific gap that remains unfilled in the literature.

2.1 Federated Learning Foundations

McMahan et al. [1] introduced Federated Averaging (FedAvg), which remains the dominant FL algorithm. In FedAvg, a central server coordinates training across clients — sending the global model out, collecting locally trained updates, averaging them, and repeating. The key insight was that local SGD for multiple epochs before aggregation is both communication-efficient and surprisingly effective. FedAvg set the standard that all subsequent FL work builds on, and it is the centralised baseline this thesis compares against.

Bonawitz et al. [3] studied FedAvg at scale and were among the first to document the server’s dual role as both a coordination hub and a vulnerability. Their work identified that at production scale, the server becomes a bottleneck and its failure has system-wide consequences — an early acknowledgement of the SPOF problem that this thesis directly investigates.

2.2 Non-IID Data in Federated Learning

A key assumption in the original FedAvg paper is that data is identically distributed across clients. In practice, this is almost never true. Each device tends to have data skewed toward its own usage patterns — a user’s phone sees their photos, a hospital sees its patient population, a factory sees its own machines. This is called **non-IID data**, and it creates a problem.

When data distributions differ across clients, locally trained models diverge from each other during training. Averaging divergent models produces a global model that

does not represent any single client’s data well. This phenomenon is called **client drift**, and Li et al. [2] studied it formally, showing that non-IID conditions cause FedAvg’s convergence to slow significantly and final accuracy to drop compared to the IID case. Their analysis motivates running experiments under both data regimes — as this thesis does — to understand how each architecture handles real-world data heterogeneity.

2.3 Decentralised Federated Learning

The theoretical case for decentralised FL was established by Lian et al. [5], who proved that decentralised parallel SGD on a gossip communication graph converges at the same asymptotic rate as centralised SGD under certain conditions. This is a foundational result — it shows that removing the central server does not necessarily hurt convergence, giving theoretical credibility to the gossip-based approach.

Lalitha et al. [6] proposed a fully decentralised FL framework using gossip protocols, where each node exchanges models directly with a subset of neighbours and aggregates locally. Their work demonstrated that gossip-based FL can learn effectively without any central coordination, and they argued that the peer-to-peer structure provides inherent fault tolerance — if one node fails, the others simply continue. However, this fault tolerance claim was made theoretically, without any real deployment or failure injection to validate it.

Boyd et al. [7] provided the mathematical foundations for gossip algorithms on fixed network topologies. Their work on randomised gossip shows how information spreads through a ring topology and at what rate nodes converge to a common value. This gives the ring topology used in this thesis a solid theoretical grounding — it is not an arbitrary choice but a well-studied baseline for gossip-based distributed computation.

Positioning of Existing Work

Table 2.1 summarises key decentralised FL papers alongside this thesis. The gap is clear: no existing paper combines real infrastructure deployment with controlled fault injection.

Table 2.1: Comparison of decentralised FL works

Work	Topology	Scale	Real Infra?	Fault Tested?
Lian et al. [5]	General graph	Simulation	No	No
Lalitha et al. [6]	General gossip	Simulation	No	No
Roy et al. [9]	P2P	Simulation	No	No
Beltran et al. [8]	Survey	N/A	N/A	N/A
This thesis	Ring	8 AWS EC2	Yes	Yes

2.4 The Gap This Thesis Fills

Beltran et al. [8] conducted one of the most comprehensive surveys of decentralised FL to date. Their survey covers architectures, aggregation strategies, fault tolerance mechanisms, and open challenges across dozens of papers. A recurring observation in their survey is that decentralised FL is theoretically mature but empirically underexplored — specifically, that fault tolerance is argued for structurally but never actually demonstrated through live failure experiments on real distributed infrastructure.

Kairouz et al. [4] made a similar observation from a broader FL perspective, noting that simulation-based evaluations dominate the field and that real deployment behaviour — including failures, network variability, and hardware differences — is rarely captured in published results.

Both surveys point to the same gap: the FL community lacks empirical evidence of what decentralised FL actually does under real failure conditions, and how much accuracy it sacrifices compared to centralised FedAvg when deployed on real machines. This thesis is a direct response to that gap. It takes the architectures that the literature has proposed and analysed theoretically, implements both from scratch, runs them on real AWS EC2 infrastructure, and reports what actually happens — including under deliberate, controlled failure injection that no existing paper has performed on real hardware.

Chapter 3

Problem Formulation

3.1 Problem Statement

As established in the previous chapters, centralised federated learning has a well-known architectural weakness — everything depends on the central server. The research community has proposed decentralised FL as the solution, where nodes communicate directly in a peer-to-peer network. The theoretical case is strong: no central server means no single point of failure, and convergence proofs show that gossip-based FL can match centralised performance under ideal conditions.

But theory and simulation are not the same as real deployment. Real machines fail unexpectedly. Real networks behave differently from simulated ones. Real TCP connections time out. None of the existing decentralised FL papers have actually deployed both architectures on real infrastructure, injected failures, and measured what happens.

This thesis addresses that directly. The central research question is:

How does decentralised synchronous gossip FL compare to centralised FedAvg in terms of model accuracy, communication behaviour, and fault tolerance — when both are deployed on real cloud infrastructure under identical conditions, including under live failure injection?

3.2 Research Questions

The central question above breaks down into four specific questions that the experiments in this thesis are designed to answer.

RQ1: Does decentralised gossip FL achieve comparable accuracy to centralised FedAvg?

Removing the central server changes how models are aggregated. In centralised FL, every client’s model is averaged together in one step. In gossip FL, each node only blends with its two immediate neighbours, and knowledge spreads gradually across

the ring. This raises the question of whether this local blending is good enough to reach the same accuracy as global averaging. We test this under both IID and Non-IID data conditions, since the answer may differ depending on how similar or different the data across nodes is.

RQ2: How does each architecture behave under Non-IID data?

In practice, data across FL nodes is rarely uniform. Each node tends to have data skewed toward certain patterns — a hospital sees certain patient types, a factory sees certain machine readings. This non-IID condition is known to hurt FL accuracy. The question is whether it hurts both architectures equally, or whether the smaller aggregation pool in gossip FL (2 neighbours vs. all 7 clients) makes it more sensitive to data heterogeneity.

RQ3: What actually happens when a critical node fails during training?

This is the core fault tolerance question. In centralised FL, the server failing should halt all clients — but by exactly how much, for exactly how long, and with what final accuracy impact? In decentralised FL, a node failing should allow others to continue — but does the ring recover cleanly, or are there secondary effects? We answer this by deliberately killing the server (centralised) and one ring node (decentralised) mid-experiment on real AWS infrastructure and observing the outcome in both cases.

RQ4: Are there failure behaviours in ring gossip that theory does not predict?

Theoretical gossip models assume clean message passing — a node either sends its model successfully or it does not. Real TCP communication is more complex. Timeouts, retries, and synchronisation delays can create second-order effects that simple theoretical models miss. This question asks whether deploying on real infrastructure reveals any such behaviours.

3.3 Design Decisions

Two design choices were made deliberately to ensure the comparison is fair and interpretable.

Synchronous protocol for both architectures. In a synchronous protocol, every node waits for its partners to finish before proceeding to the next round. Using async on one side would introduce model staleness as a second variable on top of topology, making it impossible to attribute any accuracy difference to topology alone. Keeping both sides synchronous means the only thing that differs between the two systems is whether there is a central server or not.

Fixed ring topology for the decentralised system. Each node in the ring connects to exactly two neighbours — the node to its left and the node to its right. This is the standard baseline gossip topology in the literature [7], and is well-

understood in terms of how information propagates. More complex topologies — random gossip, dynamic graphs — would add variables that make the results harder to interpret. The ring is the right choice for an empirical comparison study whose goal is to isolate the effect of decentralisation itself.

3.4 Scope and Limitations

This thesis focuses specifically on synchronous FL on a fixed ring topology with 8 nodes. It does not address asynchronous FL, dynamic topologies, Byzantine fault tolerance, or large-scale deployments beyond 8 nodes. These are all valid research directions but are explicitly out of scope here — the goal is a clean, controlled comparison, not a comprehensive system covering all possible scenarios. Scaling behaviour, adaptive timeout mechanisms, and random gossip topologies are left as directions for future work.

Chapter 4

System Design and Architecture

This chapter describes the design of the two federated learning systems built for this thesis — a centralised FedAvg system and a decentralised gossip FL system. Both were implemented from scratch in Python and PyTorch using raw TCP socket communication, and deployed on real AWS EC2 instances.

4.1 Centralised FedAvg Architecture

The centralised system follows the standard FedAvg design. One node acts as the server and the remaining seven act as clients, forming a star topology where all communication passes through the server.

Each training round proceeds in three steps. First, the server opens a listener and waits to receive locally trained models from all seven clients. Once all models arrive, it computes a simple average — equal weight for each client — and produces an updated global model. Finally, it broadcasts this global model back to all clients, and the next round begins.

Communication uses two TCP ports on the server: one for clients to upload their models, and one for the server to broadcast the aggregated model back. Each model is serialised using PyTorch’s `torch.save` and sent as raw bytes over the socket. The server waits for all seven clients before aggregating — no client can proceed to the next round until the server has received from everyone and broadcast the result.

This synchronous, server-centric design is exactly the standard FedAvg setup described in McMahan et al. [1]. It serves as the baseline against which the decentralised system is compared.

The structural consequence of this design is clear: the server is the only entity that can move training forward. If it fails, all clients are left waiting indefinitely with no recovery path.

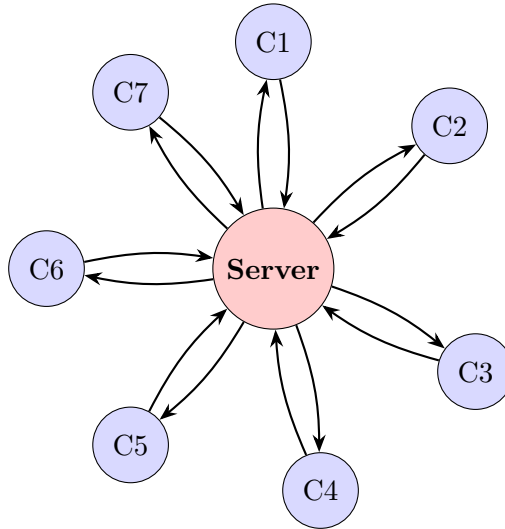


Figure 4.1: Centralised FedAvg — Star Topology (1 Server + 7 Clients)

4.2 Decentralised Gossip FL Architecture

The decentralised system eliminates the server entirely. All 8 nodes are equal peers arranged in a fixed ring topology, where each node is connected only to its immediate left and right neighbours.

Each training round proceeds in three steps as well, but the logic is local to each node. First, every node trains independently on its local data for a fixed number of epochs. Then, each node simultaneously pushes its trained model to both neighbours and receives models from both neighbours — this exchange is bidirectional and happens in parallel across the ring. Finally, each node blends the three models it now holds (its own plus the two received) using equal-weight averaging, evaluates the blended model, and waits for the next round to begin.

Communication uses one TCP port per node. Each node opens a listener for incoming models and connects outward to push its model to both neighbours. If a push fails — because the neighbour is temporarily unavailable — the node retries for up to 90 seconds before giving up and continuing with whatever models it has received. This retry mechanism is what creates the timing effects observed in the fault tolerance experiments.

Because no node plays a special role, no single node’s failure can halt the system. The remaining nodes continue exchanging with their surviving neighbours and completing training rounds independently.

4.3 Design Comparison

Table 4.1 summarises the key differences between the two architectures.

The key tradeoff is visible in the table. Centralised FL spreads knowledge instantly

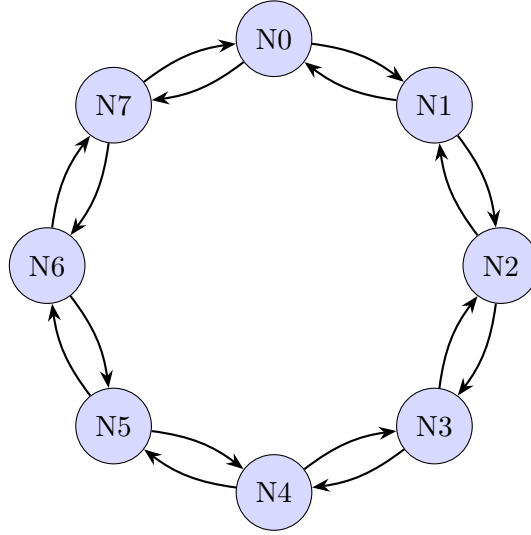


Figure 4.2: Decentralised Gossip FL — Ring Topology (8 Peer Nodes)

Table 4.1: Architecture Comparison

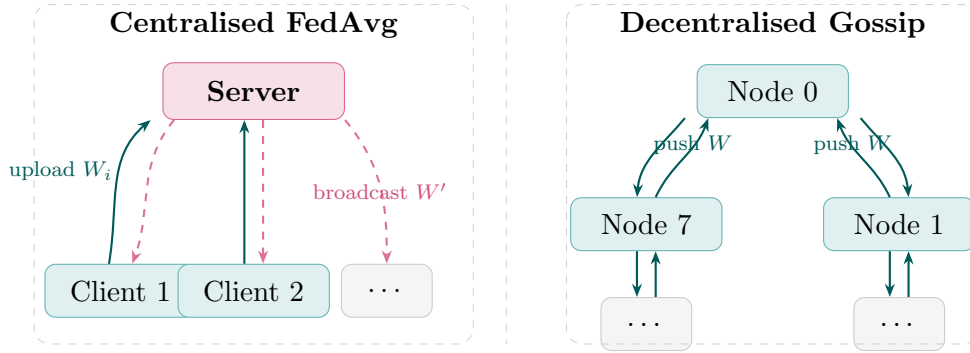
Property	Centralised FedAvg	Decentralised Gossip
Topology	Star (1 server + 7 clients)	Ring (8 equal peers)
Aggregation	Global average at server	Local blend with 2 neighbours
Communication	All clients → server	Each node ↔ 2 neighbours
SPOF	Yes (server)	No
Knowledge spread	Instant (1 round)	Gradual (across ring hops)
Failure recovery	None	Continues with available peers

— every client’s model contributes to every other client’s update in the same round. In gossip FL, knowledge spreads gradually hop by hop around the ring, which means it takes more rounds for information from one node to reach a node on the opposite side of the ring. This slower knowledge propagation is the primary reason gossip FL may achieve slightly lower accuracy, and it grows more significant under non-IID data where the information from distant nodes is particularly valuable.

4.4 Infrastructure

Both systems were deployed on AWS EC2 `t3.micro` instances running Ubuntu 22.04, all within the same VPC and subnet. Nodes communicate over the private subnet using their internal IP addresses. Each instance runs one FL process — either the server or a client in the centralised case, or one ring node in the decentralised case. No FL framework was used; both systems were implemented entirely from scratch using Python sockets and PyTorch.

4.5 Communication Protocol Detail



Centralised (left): (1) Clients upload W_i to server. (2) Server computes FedAvg. (3) Server broadcasts W' to all clients. Steps are sequential — server failure halts all clients.

Decentralised (right): (1) Each node trains locally. (2) Pushes model to both ring neighbours. (3) Receives and blends. All steps happen simultaneously — no server, no SPOF.

Figure 4.3: Communication protocol for one training round.

In the centralised system, each round involves two sequential network phases: all clients upload, then the server broadcasts. In the decentralised system, both push and receive happen in parallel across the ring — each node pushes to two neighbours and receives from two neighbours simultaneously. This parallel exchange is why gossip communication time is dominated by neighbour training speed rather than network bandwidth.

Model serialisation in both systems uses PyTorch’s `state_dict` serialised with `torch.save` to a byte buffer, transmitted as raw bytes over TCP. The serialised CNNCifar model is consistently 245.3 KB per transfer. No compression or differential encoding is used — the full model is exchanged every round.

Chapter 5

Experimental Methodology

This chapter describes the experimental setup used to compare the two architectures. It covers the infrastructure, the model and dataset, the data distribution conditions, and the six experiments that were run.

5.1 Infrastructure

Both architectures were deployed on **8 AWS EC2 t3.micro instances**, each running Ubuntu 22.04. All instances were placed within the same Virtual Private Cloud (VPC) and subnet, communicating over private IP addresses. Each instance ran a single FL process — either the server or a client in the centralised setup, or one ring node in the decentralised setup.

No FL framework was used. Both systems were implemented from scratch in Python using raw TCP sockets and PyTorch. This was a deliberate choice — using a framework would abstract away the network behaviour and make it harder to observe real failure effects.

5.2 Model and Dataset

All experiments used **CIFAR-10**, a standard image classification dataset with 50,000 training images and 10,000 test images across 10 classes. The model was a simple CNN (referred to as CNNCifar) with two convolutional layers followed by two fully connected layers, totalling approximately 62,000 parameters. The serialised model size was 245.3 KB per exchange.

CIFAR-10 was chosen because it is a well-established FL benchmark, large enough to show meaningful accuracy differences across data distributions, and small enough to run efficiently on t3.micro instances without GPU acceleration.

5.3 Data Distribution

Each experiment was run under two data distribution conditions.

IID — the 50,000 training images were split randomly and uniformly across all 7 clients (centralised) or 8 nodes (decentralised), giving each participant roughly 6,250 samples with balanced class representation. This is the ideal case where all participants see similar data.

Non-IID — data was distributed using a **Dirichlet distribution with $\alpha = 0.5$** . This causes each participant to receive a skewed sample dominated by only a few classes, making local models diverge during training. This is closer to real-world FL deployments where each device naturally sees only the data relevant to its own context. The value $\alpha = 0.5$ represents moderate heterogeneity — severe enough to meaningfully stress-test both architectures, but not so extreme as to make learning practically impossible.

5.4 Hyperparameters

The following hyperparameters were kept identical across all six experiments to ensure a fair comparison between the two architectures.

Table 5.1: Hyperparameters used across all experiments

Parameter	Value
FL rounds	50
Local epochs	5
Batch size	64
Optimiser	SGD
Learning rate	0.01
Momentum	0.5
Inter-round sleep	5 seconds

5.5 Pilot Study

Before the main AWS experiments, a two-node pilot study was conducted on two VMware virtual machines connected over a bridged local network. This pilot confirmed that the P2P TCP communication infrastructure worked correctly across real machines, and produced an unplanned server crash at Round 17 of the centralised IID experiment. That observation — the server failing and the client being left with no recovery path — directly motivated the deliberate fault injection experiments (Experiments 5-A and 5-B) in the main AWS study.

The pilot results are not included in the primary comparison because the two-node setup is a degenerate case: at $N=2$, centralised FedAvg and decentralised gossip are mathematically identical, both computing the average of exactly two models. The topology makes no difference to accuracy at this scale.

5.6 The Six Experiments

Table 5.2 lists all six experiments. Experiments 1–4 evaluate normal training behaviour under both data conditions. Experiments 5-A and 5-B evaluate fault behaviour through deliberate failure injection after Round 10.

Table 5.2: Summary of all six experiments

Exp	Architecture	Data	Purpose
1	Centralised FedAvg	IID	Accuracy baseline
2	Centralised FedAvg	Non-IID	Effect of data heterogeneity
3	Decentralised Gossip	IID	Compare accuracy against Exp 1
4	Decentralised Gossip	Non-IID	Compare accuracy against Exp 2
5-A	Decentralised Gossip	Non-IID	Node 3 exits after Round 10
5-B	Centralised FedAvg	Non-IID	Server exits after Round 10

For Experiments 5-A and 5-B, the failure was injected by manually terminating the process on the target instance after Round 10 completed. In 5-A, Node 3 (one of the eight ring nodes) was killed. In 5-B, the server instance was killed. Both experiments used Non-IID data so that the failure effects could be observed against the more challenging baseline condition.

The outcome of interest in the fault experiments was not just whether the remaining nodes continued — but exactly how they continued: which nodes were affected, by how much, over how many rounds, and whether any secondary effects appeared beyond the direct neighbours of the failed node.

Chapter 6

Results and Discussion

This chapter presents the results of all six experiments. Sections 6.2 and 6.3 cover normal training under IID and Non-IID conditions for each architecture. Section 6.4 compares both architectures directly. Section 6.5 presents the fault injection experiments.

6.1 Pilot Study: 2-VM VMware Results

Before the main AWS experiments, a two-node pilot study on VMware virtual machines validated the P2P communication infrastructure and produced the motivating server crash observation described in Chapter 5. The results are reported here briefly for completeness. They are **not** included in the primary comparison — at $N=2$, centralised FedAvg and decentralised gossip are mathematically identical, so no architectural difference in accuracy is expected or observed.

Both architectures were run on two Ubuntu VMs (192.168.221.128 and 192.168.221.129) using VMware bridged networking. The same CNNCifar model, CIFAR-10 dataset (500 samples per node), and hyperparameters were used. Both ran for 30 rounds.

IID result: Both architectures reached identical final accuracy of 36.68% at Round 30 on both VMs. The decentralised system’s per-round communication time averaged ~ 2.0 s versus ~ 33 s for centralised — a $16\times$ difference arising from eliminating the sequential server aggregation cycle.

Non-IID result ($\alpha = 0.1$): Again identical final accuracy of 28.22% on both VMs. Client drift was observed in both architectures — Node 1 (VM2) was consistently hurt by aggregation in 27 of 30 rounds due to its skewed class distribution, confirming that drift is a property of data heterogeneity, not topology.

Motivating observation: During the IID centralised experiment, an unplanned server broadcast failure at Round 17 caused Client 1 (VM2) to permanently disconnect with no recovery. This live SPOF observation directly motivated Experiment 5-B, where the same failure mode was deliberately injected on the AWS infrastructure.

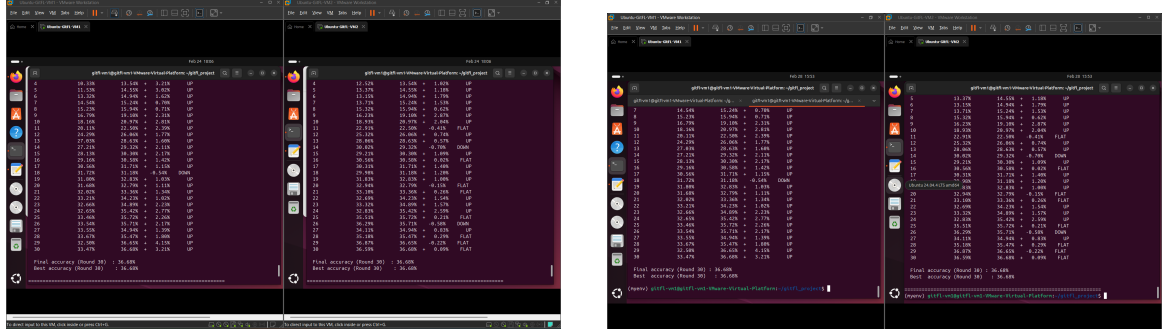


Figure 6.1: 2-VM pilot — IID results. Left: decentralised (both VMs complete 30/30 rounds, 36.68%). Right: centralised (VM2 disconnects at Round 17 due to server broadcast failure — the motivating SPOF observation for Experiment 5-B).

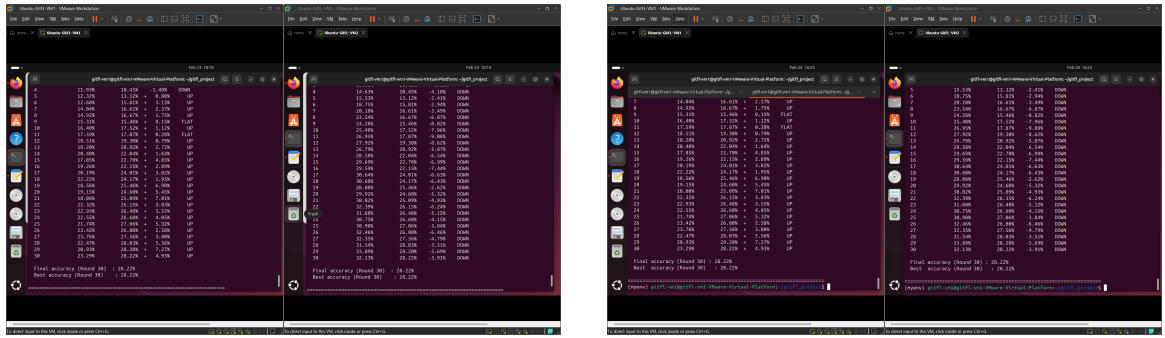


Figure 6.2: 2-VM pilot — Non-IID $\alpha=0.1$ results. Identical final accuracy (28.22%) in both architectures. Client drift (DOWN trend for Node 1/Client 1) is identical in both, confirming implementation correctness.

6.2 Centralised FedAvg: Experiments 1 and 2

6.2.1 IID Results (Experiment 1)

Under IID conditions, centralised FedAvg converges rapidly and consistently. The global model crosses 50% accuracy by Round 8 (51.50%), reaches 58% by Round 16, and peaks at **61.69%** at Round 28. The final accuracy at Round 50 is 60.02%, reflecting a mild post-peak drift of 1.67 percentage points as the model oscillates around a local optimum.

Because data is uniformly distributed across all 7 clients, local training already produces models that generalise reasonably well. FedAvg provides a modest +1 to +4 pp boost per round. The local and aggregated accuracy curves track closely throughout training, confirming that IID clients are learning approximate global representations independently.

```

ubuntu@ip-172-31-21-108:~$
[14:45:35] Sent to client 5/7 | Node 1 (172.31.31.28) | 245.3 KB
[14:45:36] Sent to client 6/7 | Node 7 (172.31.18.64) | 245.3 KB
[14:45:36] Sent to client 7/7 | Node 5 (172.31.22.247) | 245.3 KB

[14:45:36] Broadcast complete | Sent to 7/7 clients | Total: 1717.3 KB | Time: 2.981s
[14:45:36] Waiting 5s before next round...

=====
ROUND 2 / 50 -- SERVER -- 2026-05-07 14:45:41
=====

[1/3] RECEIVING MODELS FROM ALL CLIENTS

[14:45:41] Opening listener on 172.31.21.108:9000
[14:45:41] Waiting for 7 clients to upload their trained models...

[14:45:56] Received from client 1/7 | Node 2 (172.31.24.251) | 245.3 KB
[14:45:56] Received from client 2/7 | Node 4 (172.31.24.136) | 245.3 KB
[14:45:57] Received from client 3/7 | Node 6 (172.31.20.96) | 245.3 KB
[14:45:57] Received from client 4/7 | Node 3 (172.31.26.122) | 245.3 KB

[14:45:57] Done | Time: 11.776s | Accuracy: 30.46%

=====
[2/3] UPLOADING TO SERVER (172.31.21.108:9000)

[14:45:57] Model size: 245.3 KB | Connecting...
[14:45:57] Upload complete | Time: 0.000s

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[14:45:57] Waiting for server broadcast...
[14:45:57] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining

=====
ip-172-31-21-108 14:45 07-May-26
=====

ubuntu@ip-172-31-20-96:~$
[14:45:47] Epoch 2/5 | Loss: 2.2028 | Train Acc: 19.71%
[14:45:49] Epoch 3/5 | Loss: 2.0807 | Train Acc: 24.54%
[14:45:52] Epoch 4/5 | Loss: 1.9841 | Train Acc: 28.48%
[14:45:54] Epoch 5/5 | Loss: 1.9262 | Train Acc: 30.90%

[14:45:57] Done | Time: 11.839s | Accuracy: 29.87%

=====
[2/3] UPLOADING TO SERVER (172.31.21.108:9000)

[14:45:57] Model size: 245.3 KB | Connecting...
[14:45:57] Upload complete | Time: 0.001s

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[14:45:57] Waiting for server broadcast...
[14:45:57] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining

=====
ip-172-31-20-96 14:45 07-May-26
=====

ubuntu@ip-172-31-26-122:~$
[14:45:47] Epoch 2/5 | Loss: 2.1925 | Train Acc: 20.06%
[14:45:49] Epoch 3/5 | Loss: 2.0688 | Train Acc: 25.30%
[14:45:52] Epoch 4/5 | Loss: 1.9784 | Train Acc: 28.56%
[14:45:54] Epoch 5/5 | Loss: 1.9215 | Train Acc: 30.53%

[14:45:57] Done | Time: 11.776s | Accuracy: 30.46%

=====
[2/3] UPLOADING TO SERVER (172.31.21.108:9000)

[14:45:57] Model size: 245.3 KB | Connecting...
[14:45:57] Upload complete | Time: 0.000s

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[14:45:57] Waiting for server broadcast...
[14:45:57] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining

=====
ip-172-31-26-122 14:45 07-May-26
=====

ubuntu@ip-172-31-31-28:~$
[14:45:11] Download attempt 9 failed | [Errno 111] Connection refused | 275s remaining
[14:45:14] Download attempt 10 failed | [Errno 111] Connection refused | 272s remaining
[14:45:17] Download attempt 11 failed | [Errno 111] Connection refused | 269s remaining
[14:45:20] Download attempt 12 failed | [Errno 111] Connection refused | 266s remaining
[14:45:23] Download attempt 13 failed | [Errno 111] Connection refused | 263s remaining
[14:45:26] Download attempt 14 failed | [Errno 111] Connection refused | 260s remaining
[14:45:29] Download attempt 15 failed | [Errno 111] Connection refused | 257s remaining
[14:45:32] Download attempt 16 failed | [Errno 111] Connection refused | 254s remaining
[14:45:38] Received: 245.3 KB | Upload: 0.001s | Download: 48.013s | Total comm: 48.014s
[14:45:38] Accuracy local: 14.59% --> aggregated: 15.55% (change: +0.96%)

[14:45:38] Waiting 5s before next round...

=====
ROUND 2 / 50 -- Node 1 -- 2026-05-07 14:45:43
=====

[14:45:43] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[14:45:45] Epoch 1/5 | Loss: 2.2707 | Train Acc: 17.42%
[14:45:48] Epoch 2/5 | Loss: 2.1809 | Train Acc: 20.27%
[14:45:50] Epoch 3/5 | Loss: 2.0805 | Train Acc: 26.16%
[14:45:53] Epoch 4/5 | Loss: 1.9722 | Train Acc: 28.77%
[14:45:55] Epoch 5/5 | Loss: 1.9181 | Train Acc: 30.77%

=====
ip-172-31-31-28 14:45 07-May-26
=====

```

Figure 6.3: Centralized FedAvg — IID experiment, early training (Round 2). All 7 clients upload to the server; server broadcasts the aggregated model.

```

ubuntu@ip-172-31-21-108:~$
[15:05:26] Sent to client 1/7 | Node 3 (172.31.26.122) | 245.3 KB
[15:05:27] Sent to client 2/7 | Node 4 (172.31.24.136) | 245.3 KB
[15:05:27] Sent to client 3/7 | Node 7 (172.31.18.64) | 245.3 KB
[15:05:27] Sent to client 4/7 | Node 6 (172.31.20.96) | 245.3 KB
[15:05:28] Sent to client 5/7 | Node 5 (172.31.22.247) | 245.3 KB
[15:05:28] Sent to client 6/7 | Node 1 (172.31.31.28) | 245.3 KB
[15:05:29] Sent to client 7/7 | Node 2 (172.31.24.251) | 245.3 KB

[15:05:29] Broadcast complete | Sent to 7/7 clients | Total: 1717.3 KB | Time: 2.996s
[15:05:29] Waiting 5s before next round...

=====
ROUND 48 / 50 -- SERVER -- 2026-05-07 15:05:34
=====

[1/3] RECEIVING MODELS FROM ALL CLIENTS

[15:05:34] Opening listener on 172.31.21.108:9000
[15:05:34] Waiting for 7 clients to upload their trained models...

[15:05:34] Received: 245.3 KB | Upload: 0.000s | Download: 3.002s | Total comm: 3.003s
[15:05:34] Accuracy local: 57.61% --> aggregated: 60.34% (change: +2.73%)

[15:05:34] Waiting 5s before next round...

=====
ip-172-31-21-108 15:05 07-May-26
=====

ubuntu@ip-172-31-20-96:~$
[15:05:24] Upload complete | Time: 0.001s

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[15:05:24] Waiting for server broadcast...
[15:05:24] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining
[15:05:30] Received: 245.3 KB | Upload: 0.001s | Download: 3.003s | Total comm: 3.003s
[15:05:30] Accuracy local: 57.60% --> aggregated: 60.34% (change: +2.74%)

[15:05:30] Waiting 5s before next round...

=====
ROUND 48 / 50 -- Node 6 -- 2026-05-07 15:05:35
=====

[15:05:35] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[15:05:38] Epoch 1/5 | Loss: 0.6661 | Train Acc: 77.10%
[15:05:40] Epoch 2/5 | Loss: 0.2754 | Train Acc: 90.64%

=====
ip-172-31-20-96 15:05 07-May-26
=====

ubuntu@ip-172-31-26-122:~$
[15:05:23] Upload complete | Time: 0.000s

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[15:05:23] Waiting for server broadcast...
[15:05:23] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining
[15:05:29] Received: 245.3 KB | Upload: 0.000s | Download: 3.002s | Total comm: 3.003s
[15:05:29] Accuracy local: 57.61% --> aggregated: 60.34% (change: +2.73%)

[15:05:29] Waiting 5s before next round...

=====
ROUND 48 / 50 -- Node 3 -- 2026-05-07 15:05:34
=====

[15:05:34] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[15:05:36] Epoch 1/5 | Loss: 0.6744 | Train Acc: 77.86%
[15:05:38] Epoch 2/5 | Loss: 0.2746 | Train Acc: 90.16%
[15:05:41] Epoch 3/5 | Loss: 0.1408 | Train Acc: 96.02%

=====
ip-172-31-26-122 15:05 07-May-26
=====

ubuntu@ip-172-31-31-28:~$
[15:05:25] Upload complete | Time: 0.000s

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[15:05:25] Waiting for server broadcast...
[15:05:25] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining
[15:05:31] Received: 245.3 KB | Upload: 0.000s | Download: 3.002s | Total comm: 3.002s
[15:05:31] Accuracy local: 58.42% --> aggregated: 60.34% (change: +1.92%)

[15:05:31] Waiting 5s before next round...

=====
ROUND 48 / 50 -- Node 1 -- 2026-05-07 15:05:36
=====

[15:05:36] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[15:05:39] Epoch 1/5 | Loss: 0.6323 | Train Acc: 78.77%
[15:05:41] Epoch 2/5 | Loss: 0.2800 | Train Acc: 90.16%

=====
ip-172-31-31-28 15:05 07-May-26
=====

```

Figure 6.4: Centralized FedAvg — IID experiment, late training (Round 48). Convergence is stable with all clients reaching 60%+ local accuracy.

6.2.2 Non-IID Results (Experiment 2)

Under Non-IID conditions (Dirichlet $\alpha = 0.5$), the peak global accuracy drops to **57.60%** at Round 26 — a reduction of 4.09 percentage points from the IID peak. The final accuracy at Round 50 is 56.58%.

The more striking effect is the gap between local and aggregated accuracy. In steady state, clients’ local accuracy plateaus between 39–47%, while the aggregated global model reaches 56–58%. This persistent gap of 10–12 pp arises because each client only observes 2–4 dominant classes locally; the global model from FedAvg compensates for this by averaging across clients with complementary class distributions. The per-round FedAvg boost is correspondingly large — ranging from +6 to +28 pp per round compared to +1 to +4 pp under IID. Client 3 at Round 23 illustrates this most starkly: its local accuracy was 28.58%, yet the aggregated model reached 57.39% — a +28.81 pp boost from global averaging alone.

Table 6.1 quantifies the FedAvg boost for a representative Non-IID client (Client 1) at selected rounds, illustrating how global aggregation compensates for local data specialisation.

Table 6.1: Local vs. aggregated accuracy for Client 1 under Non-IID (Exp 2). FedAvg boost = aggregated – local.

Round	Local Acc	Aggregated Acc	FedAvg Boost
1	10.24%	14.89%	+4.65 pp
5	29.47%	41.91%	+12.44 pp
10	34.16%	51.20%	+17.04 pp
20	40.29%	56.23%	+15.94 pp
30	43.11%	57.08%	+13.97 pp
50	44.88%	56.58%	+11.70 pp

The boost is largest in the middle rounds (10–20) where the global model has already converged significantly while local training remains constrained by the client’s narrow class distribution. Even at Round 50, FedAvg contributes a persistent +11.70 pp over what the client could achieve locally — confirming that global aggregation is indispensable under Non-IID data.

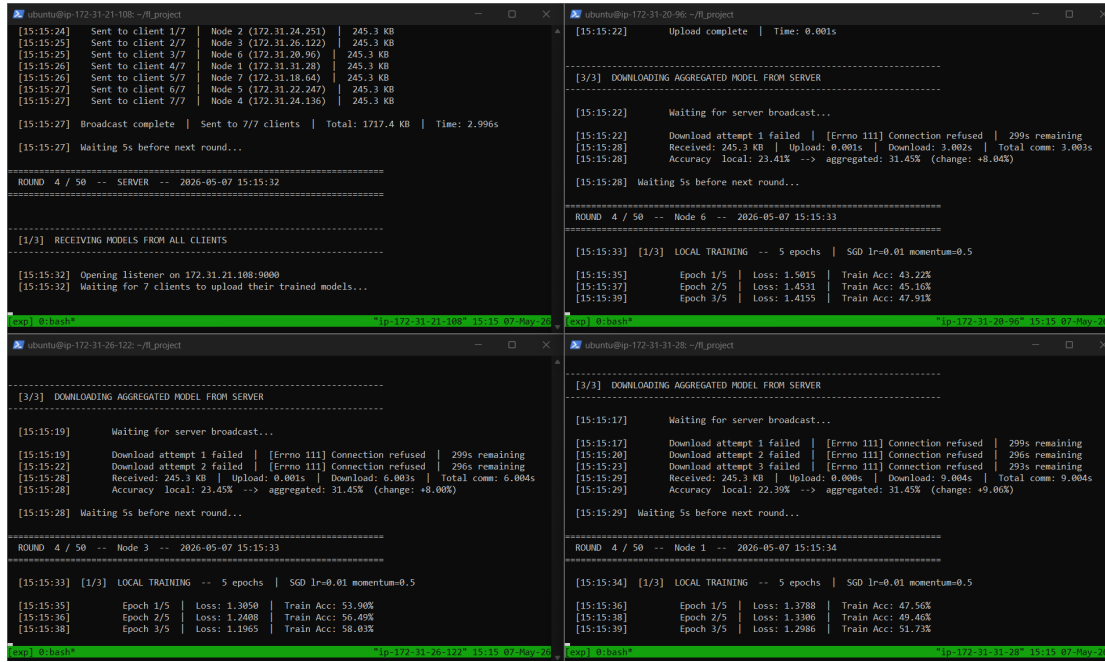


Figure 6.5: Centralized FedAvg — Non-IID experiment, early training (Round 4). Large per-round FedAvg boosts visible as clients’ local accuracy is far below the aggregated global accuracy.

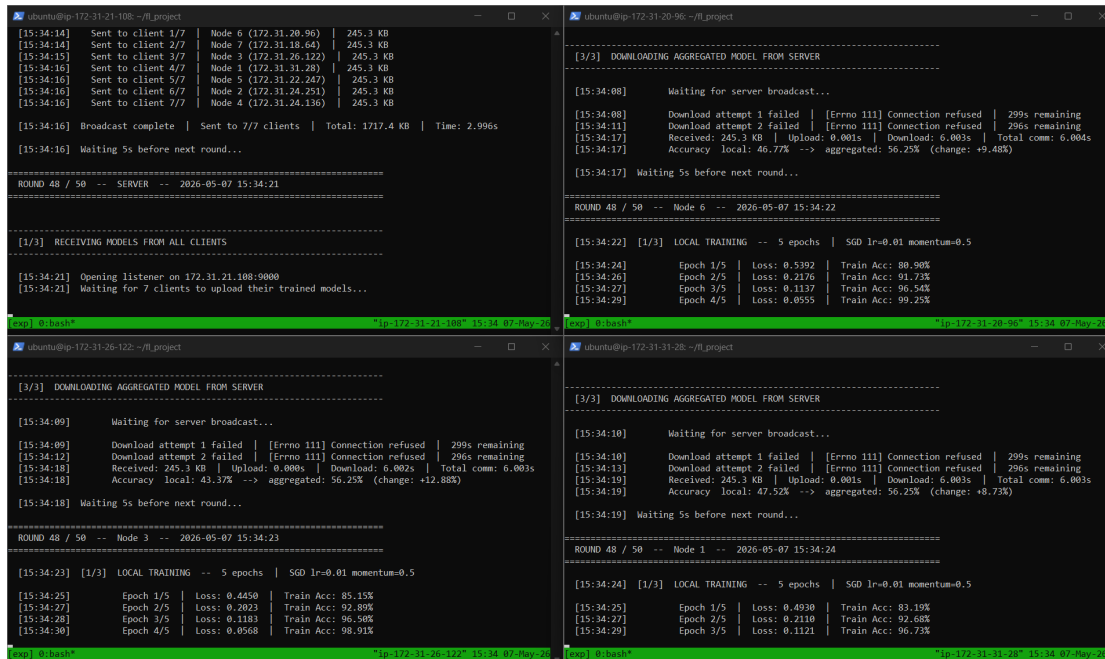


Figure 6.6: Centralized FedAvg — Non-IID experiment, late training (Round 48). Aggregated accuracy stabilises around 56–58% despite local accuracy remaining well below that range.

Table 6.2 summarises the convergence milestones for both centralised experiments.

Table 6.2: Convergence milestones — Centralised FedAvg

Milestone	IID Round	IID Acc	Non-IID Round	Non-IID Acc
Crosses 40%	R4	42.31%	R5	41.91%
Crosses 50%	R8	51.50%	R10	51.20%
Crosses 55%	R13	57.73%	R15	55.02%
Peak	R28	61.69%	R26	57.60%
Final (R50)	—	60.02%	—	56.58%

Figure 6.7 plots the global accuracy per round for both centralised experiments across all 50 rounds.

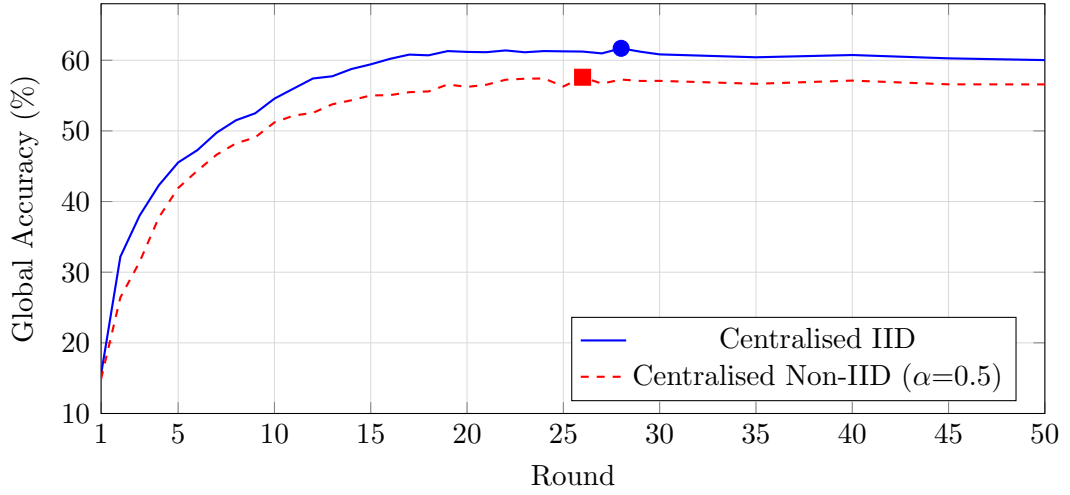


Figure 6.7: Global accuracy vs. round for Experiments 1 and 2 (Centralised FedAvg). IID peaks at 61.69% (Round 28); Non-IID peaks at 57.60% (Round 26). The gap stabilises at 3.4–5.1 pp from Round 12 onward. Filled markers indicate peak accuracy points.

Both experiments ran for approximately 22 minutes and transferred an identical 171,726–171,734 KB of data over 50 rounds. The data distribution has no impact on communication cost in centralised FedAvg, since model size is fixed regardless of local class distribution.

A structural observation from both experiments: every client’s upload phase blocks until all 7 clients have connected, and every download phase blocks until the server broadcasts. During the 5-second inter-round wait, all 7 clients log a failed connection attempt before successfully reconnecting. This synchronous server dependency is the foundation of the SPOF risk demonstrated in Experiment 5-B.

6.3 Decentralised Gossip FL: Experiments 3 and 4

6.3.1 IID Results (Experiment 3)

Under IID conditions, decentralised ring gossip converges to an average final accuracy of **55.98%** across 8 nodes (range: 55.37–56.79%). All 8 nodes self-coordinated without any central server and completed 50/50 rounds. No orchestration service or shared coordinator was involved.

The gossip blend provides a consistent +2 to +4 pp boost over each node’s local training accuracy per round. The spread in final accuracy across nodes is only **1.42 pp** — approaching but not reaching the perfect uniformity guaranteed by centralised FedAvg, where all clients receive the same server-broadcast model each round.

Occasional negative blend events were observed — rounds where blending with two neighbours briefly decreased accuracy (Round 3: -2.99 pp). These are short-lived and the model recovers within 1–2 rounds. They occur when neighbours’ models are weakly trained early in the experiment and carry noise rather than complementary knowledge. This higher round-to-round variance compared to FedAvg is an inherent characteristic of averaging within a small 2-neighbour neighbourhood.

```

[16:17:28] PUSH retry 8 --> Node-1 | [Errno 111] Connection refused | 75s left
[16:17:30] PUSH retry 9 --> Node-1 | [Errno 111] Connection refused | 73s left
[16:17:31] RECV OK <-- 172.31.31.28 | 245.3 KB | 2/2
[16:17:32] PUSH retry 10 --> Node-7 | [Errno 111] Connection refused | 71s left
[16:17:32] PUSH OK --> Node-1 (172.31.31.28:8001) | 245.3 KB | attempt 10
[16:17:34] PUSH retry 11 --> Node-7 | [Errno 111] Connection refused | 69s left
[16:17:35] PUSH retry 12 --> Node-7 | [Errno 111] Connection refused | 67s left
[16:17:38] PUSH OK --> Node-7 (172.31.18.64:8007) | 245.3 KB | attempt 13
[16:17:38] Push done | Left: OK | Right: OK
[16:17:38] Exchange done | Total comm time: 24.011s | Got 2/2 neighbor models

[3/3] BLEND + EVALUATE

[16:17:38] Blended 2 neighbor model(s) | Equal-weight avg (3 models) | Total blends: 8
[16:17:41] Post-train accuracy : 41.92%
[16:17:41] Post-blend accuracy : 42.50%
[16:17:41] Change : +0.58%
[16:17:41] Waiting 5s before next round...

[16:17:33] Epoch 4/5 | Loss: 1.4318 | Train Acc: 48.67%
[16:17:36] Epoch 5/5 | Loss: 1.4116 | Train Acc: 48.58%
[16:17:38] Done | Time: 11.664s | Accuracy: 44.72%

[2/3] EXCHANGE WITH RING NEIGHBORS

[16:17:38] Left (Node 2) : 172.31.24.251:8002
[16:17:38] Right (Node 4) : 172.31.24.136:8004
[16:17:38] Model serialized: 245.3 KB
[16:17:38] Opening receive listener on port 8003...
[16:17:38] PUSH retry 1 --> Node-2 | [Errno 111] Connection refused | 89s left
[16:17:38] PUSH retry 1 --> Node-4 | [Errno 111] Connection refused | 89s left
[16:17:40] PUSH retry 2 --> Node-2 | [Errno 111] Connection refused | 87s left
[16:17:40] PUSH retry 2 --> Node-4 | [Errno 111] Connection refused | 87s left
[16:17:42] PUSH retry 3 --> Node-2 | [Errno 111] Connection refused | 85s left
[16:17:42] PUSH retry 3 --> Node-4 | [Errno 111] Connection refused | 85s left
[16:17:44] PUSH retry 4 --> Node-2 | [Errno 111] Connection refused | 83s left
[16:17:44] PUSH retry 4 --> Node-4 | [Errno 111] Connection refused | 83s left

[16:17:24] PUSH retry 6 --> Node-7 | [Errno 111] Connection refused | 79s left
[16:17:26] PUSH retry 7 --> Node-7 | [Errno 111] Connection refused | 77s left
[16:17:28] PUSH retry 8 --> Node-7 | [Errno 111] Connection refused | 75s left
[16:17:30] PUSH retry 9 --> Node-7 | [Errno 111] Connection refused | 73s left
[16:17:32] PUSH retry 10 --> Node-7 | [Errno 111] Connection refused | 71s left
[16:17:34] PUSH retry 11 --> Node-7 | [Errno 111] Connection refused | 69s left
[16:17:36] PUSH retry 12 --> Node-7 | [Errno 111] Connection refused | 67s left
[16:17:37] RECV OK <-- 172.31.18.64 | 245.3 KB | 2/2
[16:17:38] PUSH OK --> Node-7 (172.31.18.64:8007) | 245.3 KB | attempt 13
[16:17:38] Push done | Left: OK | Right: OK
[16:17:38] Exchange done | Total comm time: 24.010s | Got 2/2 neighbor models

[3/3] BLEND + EVALUATE

[16:17:38] Blended 2 neighbor model(s) | Equal-weight avg (3 models) | Total blends: 8
[16:17:41] Post-train accuracy : 41.03%
[16:17:41] Post-blend accuracy : 42.41%
[16:17:41] Change : +1.38%
[16:17:41] Waiting 5s before next round...

[16:17:19] Epoch 1/5 | Loss: 1.5506 | Train Acc: 43.47%
[16:17:21] Epoch 2/5 | Loss: 1.5242 | Train Acc: 44.61%
[16:17:24] Epoch 3/5 | Loss: 1.4899 | Train Acc: 45.28%
[16:17:26] Epoch 4/5 | Loss: 1.4660 | Train Acc: 46.35%
[16:17:29] Epoch 5/5 | Loss: 1.4492 | Train Acc: 47.66%
[16:17:31] Done | Time: 11.843s | Accuracy: 45.16%

[2/3] EXCHANGE WITH RING NEIGHBORS

[16:17:31] Left (Node 0) : 172.31.21.108:8000
[16:17:31] Right (Node 2) : 172.31.24.251:8002
[16:17:31] Model serialized: 245.3 KB
[16:17:31] Opening receive listener on port 8001...
[16:17:31] PUSH OK --> Node-2 (172.31.24.251:8002) | 245.3 KB | attempt 1
[16:17:31] PUSH OK --> Node-0 (172.31.21.108:8000) | 245.3 KB | attempt 1
[16:17:31] Push done | Left: OK | Right: OK
[16:17:32] RECV OK <-- 172.31.21.108 | 245.3 KB | 1/2
  
```

Figure 6.8: Decentralised gossip — IID experiment, Round 3. Each node pushes its model to both ring neighbours and blends three models (own + left + right).

```

[16:51:13] PUSH retry 10 --> Node-1 | [Errno 111] Connection refused | 71s left
[16:51:13] PUSH retry 10 --> Node-7 | [Errno 111] Connection refused | 71s left
[16:51:15] PUSH retry 11 --> Node-1 | [Errno 111] Connection refused | 69s left
[16:51:15] PUSH retry 11 --> Node-7 | [Errno 111] Connection refused | 69s left
[16:51:17] PUSH retry 12 --> Node-1 | [Errno 111] Connection refused | 67s left
[16:51:17] PUSH retry 12 --> Node-7 | [Errno 111] Connection refused | 67s left
[16:51:19] PUSH OK --> Node-1 (172.31.21.20:8000) | 245.3 KB | attempt 13
[16:51:21] PUSH OK --> Node-7 (172.31.18.64:8007) | 245.3 KB | attempt 14
[16:51:21] Push done | Left: OK | Right: OK

[16:51:21] Exchange done | Total comm time: 26.012s | Got 2/2 neighbor models

[3/3] BLEND + EVALUATE

[16:51:21] Blended 2 neighbor model(s) | Equal-weight avg (3 models) | Total blends: 92
[16:51:23] Post-train accuracy : 54.88%
[16:51:23] Post-blend accuracy : 56.88%
[16:51:23] Change : +2.03%

[16:51:23] Waiting 5s before next round...

[16:51:01] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5
[16:51:04] Epoch 1/5 | Loss: 0.3746 | Train Acc: 86.94%
[16:51:06] Epoch 2/5 | Loss: 0.1577 | Train Acc: 94.99%
[16:51:08] Epoch 3/5 | Loss: 0.0456 | Train Acc: 99.30%
[16:51:11] Epoch 4/5 | Loss: 0.0166 | Train Acc: 99.95%
[16:51:13] Epoch 5/5 | Loss: 0.0092 | Train Acc: 100.00%
[16:51:16] Done | Time: 11.704s | Accuracy: 53.85%

[2/3] EXCHANGE WITH RING NEIGHBORS

[16:51:16] Left (Node 2) : 172.31.24.251:8002
[16:51:16] Right (Node 4) : 172.31.24.136:8004
[16:51:16] Model serialized: 245.3 KB
[16:51:16] Opening receive listener on port 8003...
[16:51:16] PUSH OK --> Node-2 (172.31.24.251:8002) | 245.3 KB | attempt 1
[16:51:16] PUSH OK --> Node-4 (172.31.24.136:8004) | 245.3 KB | attempt 1
[16:51:16] Push done | Left: OK | Right: OK

[16:51:10] Epoch 3/5 | Loss: 0.0790 | Train Acc: 97.94%
[16:51:13] Epoch 4/5 | Loss: 0.0278 | Train Acc: 99.71%
[16:51:15] Epoch 5/5 | Loss: 0.0125 | Train Acc: 99.95%
[16:51:18] Done | Time: 12.060s | Accuracy: 54.81%

[2/3] EXCHANGE WITH RING NEIGHBORS

[16:51:18] Left (Node 0) : 172.31.21.108:8000
[16:51:18] Right (Node 2) : 172.31.24.251:8002
[16:51:18] Model serialized: 245.3 KB
[16:51:18] Opening receive listener on port 8001...
[16:51:18] PUSH retry 1 --> Node-0 | [Errno 111] Connection refused | 89s left
[16:51:18] PUSH OK --> Node-2 (172.31.24.251:8002) | 245.3 KB | attempt 1
[16:51:19] RECV OK <-- 172.31.21.108 | 245.3 KB | 1/2
[16:51:20] PUSH retry 2 --> Node-0 | [Errno 111] Connection refused | 87s left
[16:51:22] PUSH retry 3 --> Node-0 | [Errno 111] Connection refused | 85s left
[16:51:24] PUSH retry 4 --> Node-0 | [Errno 111] Connection refused | 83s left
[16:51:26] PUSH retry 5 --> Node-0 | [Errno 111] Connection refused | 81s left

```

Figure 6.9: Decentralised gossip — IID experiment, Round 46. Nodes converge within a tight accuracy band with no central coordinator.

6.3.2 Non-IID Results (Experiment 4)

Under Non-IID conditions, average final accuracy drops to **49.49%** (range: 48.00–51.33%) — a reduction of **6.49 pp** from the IID case. This degradation is substantially larger than the 3.44 pp IID-to-Non-IID gap observed in centralised FedAvg.

The reason is the smaller aggregation pool. In each round, a gossip node blends only with its 2 immediate ring neighbours. If those neighbours carry correlated class imbalances — as is likely under Non-IID when nearby ring positions share similar data characteristics — the blend offers limited complementary knowledge. FedAvg, by contrast, aggregates all 7 clients every round and sees the full diversity of class distributions regardless of data heterogeneity.

The node accuracy spread under Non-IID is **3.33 pp** at Round 50 — more than double the 1.42 pp spread under IID. Unlike centralised FL where all clients always hold the same model, gossip nodes develop slightly different models that reflect their local neighbourhood. Under Non-IID, this divergence is amplified. This is not a bug but an inherent architectural property: gossip-based FL does not guarantee model uniformity across nodes.

Despite this, the gossip blend remains valuable. Per-round blending boosts each node’s accuracy by +7 to +12 pp over its local model under Non-IID — 3 to 5 times the boost observed under IID. The local model alone is simply too specialised to reach acceptable accuracy; neighbourhood knowledge is essential.

```

[17:00:11] Epoch 5/5 | Loss: 1.1261 | Train Acc: 61.25%
[17:00:14] Done | Time: 14.679s | Accuracy: 32.26%

[2/3] EXCHANGE WITH RING NEIGHBORS

[17:00:14] Left (Node 7) : 172.31.18.64:8007
[17:00:14] Right (Node 1) : 172.31.31.28:8001

[17:00:14] Model serialized: 245.3 KB
[17:00:14] Opening receive listener on port 8000...

[17:00:14] PUSH retry 1 --> Node-1 | [Errno 111] Connection refused | 89s left
[17:00:14] PUSH retry 1 --> Node-7 | [Errno 111] Connection refused | 89s left
[17:00:14] RECV OK <-- 172.31.18.64 | 245.3 KB | 1/2
[17:00:16] PUSH retry 2 --> Node-1 | [Errno 111] Connection refused | 87s left
[17:00:16] PUSH retry 2 --> Node-7 | [Errno 111] Connection refused | 87s left
[17:00:17] RECV OK <-- 172.31.31.28 | 245.3 KB | 2/2
[17:00:18] PUSH retry 3 --> Node-1 | [Errno 111] Connection refused | 85s left
[17:00:18] PUSH OK --> Node-1 (172.31.31.28:8001) | 245.3 KB | attempt 3
[17:00:20] PUSH retry 4 --> Node-7 | [Errno 111] Connection refused | 83s left

[17:00:12] RECV OK <-- 172.31.24.136 | 245.3 KB | 2/2
[17:00:13] PUSH OK --> Node-4 (172.31.24.136:8004) | 245.3 KB | attempt 14
[17:00:13] Push done | Left: OK | Right: OK

[17:00:13] Exchange done | Total comm time: 26.015s | Got 2/2 neighbor models

[3/3] BLEND + EVALUATE

[17:00:13] Blended 2 neighbor model(s) | Equal-weight avg (3 models) | Total blends: 10
[17:00:15] Post-train accuracy : 26.47%
[17:00:15] Post-blend accuracy : 28.38%
[17:00:15] Change : +1.91%

[17:00:15] Waiting 5s before next round...

ROUND 6 / 50 -- Node 3 -- 2026-05-07 17:00:20

[17:00:20] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[17:00:06] Epoch 4/5 | Loss: 1.3311 | Train Acc: 51.15%
[17:00:08] Epoch 5/5 | Loss: 1.2984 | Train Acc: 52.23%
[17:00:11] Done | Time: 11.027s | Accuracy: 35.14%

[2/3] EXCHANGE WITH RING NEIGHBORS

[17:00:11] Left (Node 5) : 172.31.22.247:8005
[17:00:11] Right (Node 7) : 172.31.18.64:8007

[17:00:11] Model serialized: 245.3 KB
[17:00:11] Opening receive listener on port 8006...

[17:00:11] PUSH retry 1 --> Node-7 | [Errno 111] Connection refused | 89s left
[17:00:11] PUSH OK --> Node-5 (172.31.22.247:8005) | 245.3 KB | attempt 1
[17:00:12] RECV OK <-- 172.31.18.64 | 245.3 KB | 4/2
[17:00:13] PUSH retry 2 --> Node-7 | [Errno 111] Connection refused | 87s left
[17:00:15] PUSH retry 3 --> Node-7 | [Errno 111] Connection refused | 85s left
[17:00:17] PUSH retry 4 --> Node-7 | [Errno 111] Connection refused | 83s left
[17:00:19] PUSH retry 5 --> Node-7 | [Errno 111] Connection refused | 81s left
[17:00:21] PUSH retry 6 --> Node-7 | [Errno 111] Connection refused | 79s left

[17:00:07] Epoch 1/5 | Loss: 1.2400 | Train Acc: 53.91%
[17:00:09] Epoch 2/5 | Loss: 1.1817 | Train Acc: 55.03%
[17:00:11] Epoch 3/5 | Loss: 1.1600 | Train Acc: 57.29%
[17:00:13] Epoch 4/5 | Loss: 1.1264 | Train Acc: 57.90%
[17:00:15] Epoch 5/5 | Loss: 1.1033 | Train Acc: 58.41%
[17:00:17] Done | Time: 9.092s | Accuracy: 31.60%

[2/3] EXCHANGE WITH RING NEIGHBORS

[17:00:17] Left (Node 0) : 172.31.21.108:8000
[17:00:17] Right (Node 2) : 172.31.24.251:8002

[17:00:17] Model serialized: 245.3 KB
[17:00:17] Opening receive listener on port 8001...

[17:00:17] PUSH OK --> Node-2 (172.31.24.251:8002) | 245.3 KB | attempt 1
[17:00:17] PUSH OK --> Node-0 (172.31.21.108:8000) | 245.3 KB | attempt 1
[17:00:17] Push done | Left: OK | Right: OK
[17:00:18] RECV OK <-- 172.31.21.108 | 245.3 KB | 1/2

```

Figure 6.10: Decentralised gossip — Non-IID experiment, Round 8. PUSH retry attempts visible as some nodes take longer to synchronise with neighbours whose training speed differs.

```

[17:32:24] Push done | Left: OK | Right: OK
[17:32:24] Exchange done | Total comm time: 24.011s | Got 2/2 neighbor models

[3/3] BLEND + EVALUATE

[17:32:24] Blended 2 neighbor model(s) | Equal-weight avg (3 models) | Total blends: 90
[17:32:26] Post-train accuracy : 44.71%
[17:32:26] Post-blend accuracy : 51.72%
[17:32:26] Change : +7.02%

[17:32:26] Waiting 5s before next round...

ROUND 46 / 50 -- Node 0 -- 2026-05-07 17:32:31

[17:32:31] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5
[17:32:34] Epoch 1/5 | Loss: 0.3277 | Train Acc: 88.46%
[17:32:36] Epoch 2/5 | Loss: 0.1217 | Train Acc: 95.89%

[17:32:04] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5
[17:32:06] Epoch 1/5 | Loss: 0.3068 | Train Acc: 88.90%
[17:32:08] Epoch 2/5 | Loss: 0.1384 | Train Acc: 95.36%
[17:32:10] Epoch 3/5 | Loss: 0.0343 | Train Acc: 99.49%
[17:32:12] Epoch 4/5 | Loss: 0.0517 | Train Acc: 98.97%
[17:32:13] Epoch 5/5 | Loss: 0.0119 | Train Acc: 99.30%
[17:32:16] Done | Time: 9.518s | Accuracy: 41.73%

[2/3] EXCHANGE WITH RING NEIGHBORS

[17:32:16] Left (Node 2) : 172.31.24.251:8002
[17:32:16] Right (Node 4) : 172.31.24.136:8004

[17:32:16] Model serialized: 245.3 KB
[17:32:16] Opening receive listener on port 8003...

[17:32:16] PUSH OK --> Node-4 (172.31.24.136:8004) | 245.3 KB | attempt 1
[17:32:16] PUSH OK --> Node-2 (172.31.24.251:8002) | 245.3 KB | attempt 1
[17:32:16] Push done | Left: OK | Right: OK

[17:32:25] Push done | Left: OK | Right: OK
[17:32:25] Exchange done | Total comm time: 28.019s | Got 2/2 neighbor models

[3/3] BLEND + EVALUATE

[17:32:25] Blended 2 neighbor model(s) | Equal-weight avg (3 models) | Total blends: 90
[17:32:28] Post-train accuracy : 45.27%
[17:32:28] Post-blend accuracy : 52.20%
[17:32:28] Change : +6.93%

[17:32:28] Waiting 5s before next round...

ROUND 46 / 50 -- Node 6 -- 2026-05-07 17:32:33

[17:32:33] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5
[17:32:35] Epoch 1/5 | Loss: 0.5078 | Train Acc: 82.80%
[17:32:36] Epoch 2/5 | Loss: 0.1771 | Train Acc: 94.01%

[17:32:21] Done | Time: 8.960s | Accuracy: 45.64%

[2/3] EXCHANGE WITH RING NEIGHBORS

[17:32:21] Left (Node 0) : 172.31.21.108:8000
[17:32:21] Right (Node 2) : 172.31.24.251:8002

[17:32:21] Model serialized: 245.3 KB
[17:32:21] Opening receive listener on port 8001...

[17:32:21] PUSH retry 1 --> Node-0 | [Errno 111] Connection refused | 89s left
[17:32:21] PUSH OK --> Node-2 (172.31.24.251:8002) | 245.3 KB | attempt 1
[17:32:22] RECV OK <-- 172.31.21.108 | 245.3 KB | 1/2
[17:32:23] PUSH retry 2 --> Node-0 | [Errno 111] Connection refused | 87s left
[17:32:25] PUSH retry 3 --> Node-0 | [Errno 111] Connection refused | 85s left
[17:32:27] PUSH retry 4 --> Node-0 | [Errno 111] Connection refused | 83s left
[17:32:29] PUSH retry 5 --> Node-0 | [Errno 111] Connection refused | 81s left
[17:32:31] PUSH retry 6 --> Node-0 | [Errno 111] Connection refused | 79s left
[17:32:33] PUSH retry 7 --> Node-0 | [Errno 111] Connection refused | 77s left
[17:32:35] PUSH retry 8 --> Node-0 | [Errno 111] Connection refused | 75s left

```

Figure 6.11: Decentralised gossip — Non-IID experiment, Round 47. Accuracy spread across nodes is wider than under IID, reflecting the limited complementary knowledge in a 2-neighbour ring under heterogeneous data.

Table 6.3 shows per-node final accuracy for both decentralised experiments.

Table 6.3: Per-node final accuracy — Decentralised Gossip (Round 50)

Node	IID Final Acc	Non-IID Final Acc
Node 0	56.56%	48.93%
Node 1	55.89%	51.33%
Node 2	55.71%	50.11%
Node 3	56.34%	48.49%
Node 4	55.80%	49.85%
Node 5	55.64%	48.00%
Node 6	55.37%	49.60%
Node 7	56.79%	50.56%
Average	55.98%	49.49%
Spread	1.42 pp	3.33 pp

Figure 6.12 plots the post-blend accuracy per round for Node 0 (representative) in both decentralised experiments. Node 0 is representative of the network average — its final accuracy (56.17% IID, 49.16% Non-IID) is close to the 8-node averages (55.98% and 49.49% respectively).

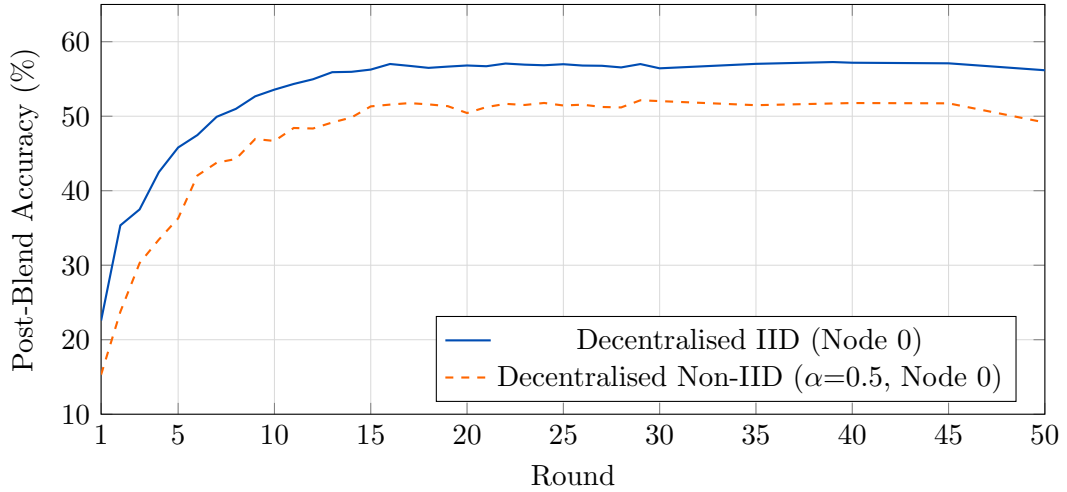


Figure 6.12: Post-blend accuracy vs. round for Experiments 3 and 4 (Decentralised Gossip, Node 0). IID converges to $\sim 57\%$ and holds steadily from Round 15. Non-IID plateaus at $\sim 51\text{--}52\%$ from Round 15, never crossing 53% sustainably. The gap between IID and Non-IID (~ 7 pp) is wider than in centralised FL (~ 3.5 pp), confirming gossip’s greater sensitivity to data heterogeneity.

Average communication time per round in the decentralised setup was $\sim 24\text{--}26$ seconds, compared to ~ 3.9 seconds in the centralised setup. This difference is not due to data volume — each node transfers approximately 981 KB per round (490.6 KB to each neighbour), roughly $2\times$ the per-client data in centralised FL. The higher

time comes from waiting for both neighbours to finish their local training before the exchange can begin. In centralised FL, the server waits for all clients; in gossip FL, each node waits for its two neighbours. Total experiment wall time was approximately 40 minutes, compared to 22.5 minutes for the centralised equivalent.

6.4 Architecture Comparison: Experiments 1–4

Table 6.4 directly compares the key results from Experiments 1–4.

Table 6.4: Architecture comparison — normal operation results

Metric	Centralised IID	Centralised Non-IID	Gossip IID	Gossip Non-IID
Peak accuracy	61.69%	57.60%	57.48%	53.44%
Final accuracy	60.02%	56.58%	55.98%	49.49%
Rounds to 50%	R8	R10	R8	R15
Accuracy spread	0 pp	0 pp	1.42 pp	3.33 pp
Avg comm/round	~3.9s	~3.9s	~25s	~24s
Nodes completed	7/7	7/7	8/8	8/8

Figure 6.13 shows how Node 2 and Node 4’s accuracy evolved across all 50 rounds in Experiment 5-A, with the three distinct phases annotated.

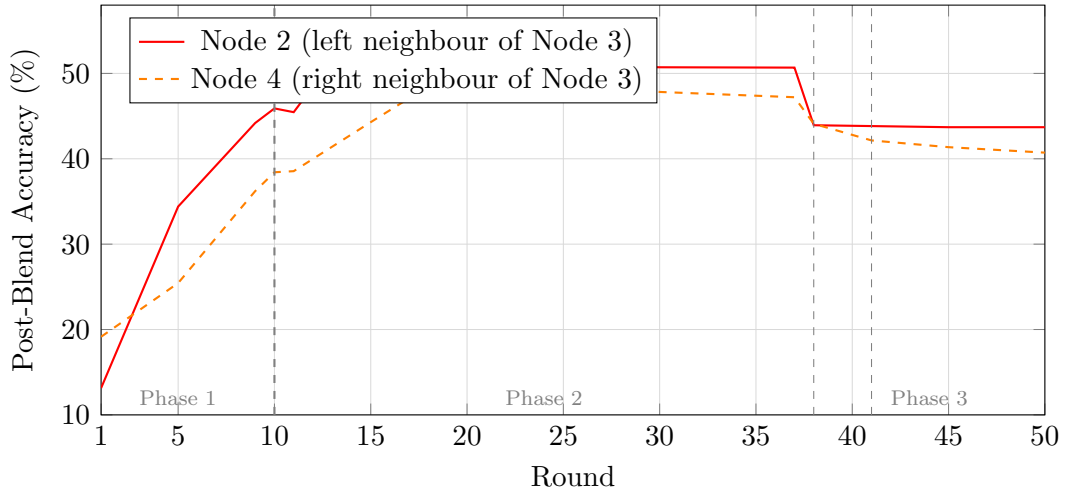


Figure 6.13: Per-round accuracy of Nodes 2 and 4 (direct neighbours of failed Node 3) in Experiment 5-A. Phase 1 (R1–10): normal operation. Phase 2 (R11–37/41): single surviving neighbour, accuracy held but comm time inflated. Phase 3 (R38/41–50): complete isolation, accuracy frozen at local training output (~43–44%).

The accuracy gap between centralised FedAvg and decentralised gossip is **4.21 pp under IID** and **7.09 pp under Non-IID** (comparing final accuracy). The gap

is larger under Non-IID because gossip’s small neighbourhood is less effective than FedAvg’s full global aggregation at overcoming class imbalance. This is the quantified accuracy cost of eliminating the central server.

Communication overhead is higher in gossip — each node waits for its neighbours to finish training, making round duration inherently tied to neighbour training speed. In centralised FL, the server aggregates instantly (0.044 seconds for FedAvg on 7 models) and communication cost is bounded by network speed. In gossip FL, communication cost is bounded by training speed.

6.5 Fault Injection: Experiments 5-A and 5-B

Both fault experiments used Non-IID data and identical model/hardware setups. In Experiment 5-A, Node 3 was killed after completing Round 10. In Experiment 5-B, the server was killed after completing Round 9’s broadcast. The outcome in each case was fundamentally different.

6.5.1 Experiment 5-B: SPOF in Centralised FL

The server exited at 19:32:26 after broadcasting the Round 9 aggregated model to all 7 clients. Within 14 seconds, all 7 clients had completed their Round 10 local training and began attempting to upload to the now-dead server.

Each client retried the upload every 3 seconds for up to 300 seconds. After 80 upload attempts over 4 minutes and 14 seconds, all 7 clients simultaneously logged `CRITICAL --- SERVER UNREACHABLE --- SPOF CONFIRMED` within a 4-second window (19:36:39–43). The system halted permanently at 9/50 rounds with a final accuracy of **49.07%**. No client could continue, recover, or route around the failure.

The potential accuracy the system would have reached with a healthy server — based on Experiment 2 — was 57.60%. The SPOF caused a permanent accuracy loss of approximately **8.5 pp** and 41 rounds of training time with no recovery path.

```

[19:30:11] Sent to client 1/7 | Node 5 (172.31.22.247) | 245.3 KB
[19:30:11] Sent to client 2/7 | Node 7 (172.31.18.64) | 245.3 KB
[19:30:12] Sent to client 3/7 | Node 3 (172.31.26.122) | 245.3 KB
[19:30:13] Sent to client 4/7 | Node 6 (172.31.20.96) | 245.3 KB
[19:30:13] Sent to client 5/7 | Node 2 (172.31.24.251) | 245.3 KB
[19:30:13] Sent to client 6/7 | Node 1 (172.31.31.28) | 245.3 KB
[19:30:13] Sent to client 7/7 | Node 4 (172.31.24.136) | 245.3 KB

[19:30:13] Broadcast complete | Sent to 7/7 clients | Total: 1717.4 KB | Time: 2.996s
[19:30:13] Waiting 5s before next round...

=====
ROUND 5 / 50 -- SERVER -- 2026-05-07 19:30:18
=====

[1/3] RECEIVING MODELS FROM ALL CLIENTS

[19:30:18] Opening listener on 172.31.21.108:9000
[19:30:18] Waiting for 7 clients to upload their trained models...

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[19:30:08] Waiting for server broadcast...

[19:30:08] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining
[19:30:14] Received: 245.3 KB | Upload: 0.001s | Download: 3.002s | Total comm: 3.003s
[19:30:14] Accuracy local: 26.65% --> aggregated: 37.56% (change: +10.91%)

[19:30:14] Waiting 5s before next round...

=====
ROUND 5 / 50 -- Node 7 -- 2026-05-07 19:30:19
=====

[19:30:19] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[19:30:22] Epoch 1/5 | Loss: 1.0156 | Train Acc: 59.63%
[19:30:24] Epoch 2/5 | Loss: 0.9685 | Train Acc: 62.39%
[19:30:26] Epoch 3/5 | Loss: 0.9479 | Train Acc: 63.14%
[19:30:28] Epoch 4/5 | Loss: 0.9167 | Train Acc: 64.95%

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[19:30:10] Waiting for server broadcast...

[19:30:10] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining
[19:30:16] Received: 245.3 KB | Upload: 0.001s | Download: 3.002s | Total comm: 3.003s
[19:30:16] Accuracy local: 25.15% --> aggregated: 37.56% (change: +12.41%)

[19:30:16] Waiting 5s before next round...

=====
ROUND 5 / 50 -- Node 3 -- 2026-05-07 19:30:20
=====

[19:30:20] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[19:30:22] Epoch 1/5 | Loss: 1.2845 | Train Acc: 58.13%
[19:30:24] Epoch 2/5 | Loss: 1.1519 | Train Acc: 59.77%
[19:30:26] Epoch 3/5 | Loss: 1.1288 | Train Acc: 60.36%
[19:30:28] Epoch 4/5 | Loss: 1.0945 | Train Acc: 61.43%

=====
[3/3] DOWNLOADING AGGREGATED MODEL FROM SERVER

[19:30:10] Waiting for server broadcast...

[19:30:10] Download attempt 1 failed | [Errno 111] Connection refused | 299s remaining
[19:30:16] Received: 245.3 KB | Upload: 0.001s | Download: 3.002s | Total comm: 3.003s
[19:30:16] Accuracy local: 25.15% --> aggregated: 37.56% (change: +12.38%)

[19:30:16] Waiting 5s before next round...

=====
ROUND 5 / 50 -- Node 4 -- 2026-05-07 19:30:21
=====

[19:30:21] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[19:30:23] Epoch 1/5 | Loss: 1.2757 | Train Acc: 58.48%
[19:30:26] Epoch 2/5 | Loss: 1.2263 | Train Acc: 52.67%
[19:30:28] Epoch 3/5 | Loss: 1.2076 | Train Acc: 53.57%

```

Figure 6.14: Centralised SPOF — Round 5, normal operation just before the server failure at Round 10.

```

* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro

System information as of Thu May 7 19:26:16 UTC 2026
System load: 0.0 Temperature: -273.1 C
Usage of /: 57.8% of 6.71GB Processes: 111
Memory usage: 25% Users logged in: 1
Swap usage: 0% IPv4 address for ens5: 172.31.21.108

* Ubuntu Pro delivers the most comprehensive open source security and
compliance features.
https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.
3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm
[exited]
ubuntu@ip-172-31-21-108:~/fl_project$

=====
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro

System information as of Thu May 7 19:27:21 UTC 2026
System load: 0.0 Temperature: -273.1 C
Usage of /: 57.8% of 6.71GB Processes: 113
Memory usage: 27% Users logged in: 1
Swap usage: 0% IPv4 address for ens5: 172.31.18.64

* Ubuntu Pro delivers the most comprehensive open source security and
compliance features.
https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.
3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm
[exited]
ubuntu@ip-172-31-18-64:~/fl_project$

=====
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro

System information as of Thu May 7 19:26:45 UTC 2026
System load: 0.0 Temperature: -273.1 C
Usage of /: 57.8% of 6.71GB Processes: 115
Memory usage: 25% Users logged in: 1
Swap usage: 0% IPv4 address for ens5: 172.31.26.122

* Ubuntu Pro delivers the most comprehensive open source security and
compliance features.
https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.
3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm
[exited]
ubuntu@ip-172-31-26-122:~/fl_project$

=====
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro

System information as of Thu May 7 19:26:55 UTC 2026
System load: 0.0 Temperature: -273.1 C
Usage of /: 57.8% of 6.71GB Processes: 112
Memory usage: 25% Users logged in: 0
Swap usage: 0% IPv4 address for ens5: 172.31.24.136

* Ubuntu Pro delivers the most comprehensive open source security and
compliance features.
https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.
3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm
[exited]
ubuntu@ip-172-31-24-136:~/fl_project$

```

Figure 6.15: AWS EC2 infrastructure showing the 4 terminal windows from the centralised SPOF experiment. After the server exits, all client terminals show repeated connection refused errors.

6.5.2 Experiment 5-A: Fault Tolerance in Decentralised FL

Node 3 completed its Round 10 blend (post-blend accuracy: 37.26%) and exited at 17:46:59. Its direct ring neighbours — Node 2 on the left and Node 4 on the right — detected the failure via push timeout at approximately 17:48:45, about 1 minute

46 seconds after the exit. Both logged PUSH FAIL → Node-3 | Neighbor may be down and continued training.

All 7 remaining nodes completed 50/50 rounds. The system never halted.

```

[17:42:52] PUSH OK --> Node-1 (172.31.31.28:8001) | 245.3 KB | attempt 11
[17:42:55] PUSH OK --> Node-7 (172.31.18.64:8007) | 245.3 KB | attempt 12
[17:42:55] Push done | Left: OK | Right: OK
[17:42:55] Exchange done | Total comm time: 22.809s | Got 2/2 neighbor models

[3/3] BLEND + EVALUATE

[17:42:55] Blended 2 neighbor model(s) | Equal-weight avg (3 models) | Total blends: 4
[17:42:58] Post-train accuracy : 22.29%
[17:42:58] Post-blend accuracy : 23.75%
[17:42:58] Change : +1.46%
[17:42:58] Waiting 5s before next round...

ROUND 3 / 50 -- Node 0 -- 2020-05-07 17:43:03

[17:43:03] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[exp] 0:hash# "ip-172-31-21-108" 17:42 07-May-20

[17:42:49] Exchange done | Total comm time: 10.804s | Got 2/2 neighbor models

[3/3] BLEND + EVALUATE

[17:42:49] Blended 2 neighbor model(s) | Equal-weight avg (3 models) | Total blends: 6
[17:42:52] Post-train accuracy : 23.65%
[17:42:52] Post-blend accuracy : 22.47%
[17:42:52] Change : -1.18%
[17:42:52] Waiting 5s before next round...

ROUND 4 / 50 -- Node 3 -- 2020-05-07 17:42:57

[17:42:57] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5

[17:42:59] Epoch 1/5 | Loss: 1.2142 | Train Acc: 57.12%
[17:43:01] Epoch 2/5 | Loss: 1.1733 | Train Acc: 59.43%
[17:43:03] Epoch 3/5 | Loss: 1.1637 | Train Acc: 59.65%

[exp] 0:hash# "ip-172-31-26-122" 17:43 07-May-20

[17:42:42] Epoch 1/5 | Loss: 1.4916 | Train Acc: 42.33%
[17:42:44] Epoch 2/5 | Loss: 1.2977 | Train Acc: 45.67%
[17:42:45] Epoch 3/5 | Loss: 1.3569 | Train Acc: 48.35%
[17:42:47] Epoch 4/5 | Loss: 1.3364 | Train Acc: 49.65%
[17:42:49] Epoch 5/5 | Loss: 1.2974 | Train Acc: 51.37%
[17:42:52] Done | Time: 9.113s | Accuracy: 27.19%

[2/3] EXCHANGE WITH RING NEIGHBORS

[17:42:52] Left (Node 0) : 172.31.21.108:8000
[17:42:52] Right (Node 2) : 172.31.24.251:8002
[17:42:52] Model serialized: 245.3 KB
[17:42:52] Opening receive listener on port 8001...
[17:42:52] PUSH OK --> Node-8 (172.31.21.108:8000) | 245.3 KB | attempt 1
[17:42:52] PUSH OK --> Node-2 (172.31.24.251:8002) | 245.3 KB | attempt 1
[17:42:52] Push done | Left: OK | Right: OK
[17:42:53] RECV OK <-- 172.31.21.108 | 245.3 KB | 1/2

[exp] 0:hash# "ip-172-31-20-96" 17:42 07-May-20
  
```

Figure 6.16: Decentralised fault tolerance — Node 3's neighbours (Nodes 2 and 4) showing PUSH retry attempts after Node 3 exits. Both continue training with their surviving neighbour.

```

[18:15:52] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5
[18:15:54] Epoch 1/5 | Loss: 0.2802 | Train Acc: 90.00%
[18:15:57] Epoch 2/5 | Loss: 0.1013 | Train Acc: 97.80%
[18:15:59] Epoch 3/5 | Loss: 0.0414 | Train Acc: 99.17%
[18:16:02] Epoch 4/5 | Loss: 0.0163 | Train Acc: 99.92%
[18:16:04] Epoch 5/5 | Loss: 0.0101 | Train Acc: 99.98%
[18:16:07] Done | Time: 12.265s | Accuracy: 44.40%

[2/3] EXCHANGE WITH RING NEIGHBORS

[18:16:07] Left (Node 7) : 172.31.18.64:8007
[18:16:07] Right (Node 1) : 172.31.31.28:8001
[18:16:07] Model serialized: 245.3 KB
[18:16:07] Opening receive listener on port 8000...
[18:16:07] PUSH OK --> Node-7 (172.31.18.64:8007) | 245.3 KB | attempt 1
[18:16:07] PUSH OK --> Node-1 (172.31.31.28:8001) | 245.3 KB | attempt 1
[18:16:07] Push done | Left: OK | Right: OK

[exp] 0:hash# "ip-172-31-21-108" 18:16 07-May-20

[18:15:52] [1/3] LOCAL TRAINING -- 5 epochs | SGD lr=0.01 momentum=0.5
[18:15:54] Epoch 1/5 | Loss: 0.4834 | Train Acc: 81.81%
[18:15:57] Epoch 2/5 | Loss: 0.2262 | Train Acc: 92.11%
[18:15:59] Epoch 3/5 | Loss: 0.1321 | Train Acc: 96.08%
[18:16:02] Epoch 4/5 | Loss: 0.0858 | Train Acc: 98.29%
[18:16:04] Epoch 5/5 | Loss: 0.0556 | Train Acc: 99.25%
[18:16:07] Done | Time: 12.236s | Accuracy: 40.42%

[2/3] EXCHANGE WITH RING NEIGHBORS

[18:16:07] Left (Node 3) : 172.31.26.122:8003
[18:16:07] Right (Node 5) : 172.31.22.247:8005
[18:16:07] Model serialized: 245.3 KB
[18:16:07] Opening receive listener on port 8004...
[18:16:07] PUSH retry 1 --> Node-3 | [Errno 111] Connection refused | 89s left
[18:16:07] PUSH OK --> Node-5 (172.31.22.247:8005) | 245.3 KB | attempt 1

[exp] 0:hash# "ip-172-31-24-136" 18:16 07-May-20

[18:14:23] Epoch 1/5 | Loss: 0.4110 | Train Acc: 86.22%
[18:14:24] Epoch 2/5 | Loss: 0.1571 | Train Acc: 94.62%
[18:14:26] Epoch 3/5 | Loss: 0.0586 | Train Acc: 98.70%
[18:14:28] Epoch 4/5 | Loss: 0.0211 | Train Acc: 99.96%
[18:14:30] Epoch 5/5 | Loss: 0.0110 | Train Acc: 100.00%
[18:14:33] Done | Time: 9.309s | Accuracy: 44.50%

[2/3] EXCHANGE WITH RING NEIGHBORS

[18:14:33] Left (Node 5) : 172.31.22.247:8005
[18:14:33] Right (Node 7) : 172.31.18.64:8007
[18:14:33] Model serialized: 245.3 KB
[18:14:33] Opening receive listener on port 8006...
[18:14:33] PUSH OK --> Node-5 (172.31.22.247:8005) | 245.3 KB | attempt 1
[18:14:33] PUSH OK --> Node-7 (172.31.18.64:8007) | 245.3 KB | attempt 1
[18:14:33] Push done | Left: OK | Right: OK
[18:14:37] RECV OK <-- 172.31.18.64 | 245.3 KB | 1/2

[exp] 0:hash# "ip-172-31-20-96" 18:15 07-May-20

[18:15:34] Epoch 1/5 | Loss: 0.2750 | Train Acc: 90.14%
[18:15:36] Epoch 2/5 | Loss: 0.1459 | Train Acc: 95.45%
[18:15:38] Epoch 3/5 | Loss: 0.0459 | Train Acc: 99.24%
[18:15:40] Epoch 4/5 | Loss: 0.0199 | Train Acc: 99.94%
[18:15:41] Epoch 5/5 | Loss: 0.0124 | Train Acc: 100.00%
[18:15:44] Done | Time: 9.019s | Accuracy: 44.54%

[2/3] EXCHANGE WITH RING NEIGHBORS

[18:15:44] Left (Node 0) : 172.31.21.108:8000
[18:15:44] Right (Node 2) : 172.31.24.251:8002
[18:15:44] Model serialized: 245.3 KB
[18:15:44] Opening receive listener on port 8001...
[18:15:44] PUSH OK --> Node-2 (172.31.24.251:8002) | 245.3 KB | attempt 1
[18:15:44] PUSH OK --> Node-8 (172.31.21.108:8000) | 245.3 KB | attempt 1
[18:15:44] Push done | Left: OK | Right: OK
[18:16:07] RECV OK <-- 172.31.21.108 | 245.3 KB | 1/2

[exp] 0:hash# "ip-172-31-31-28" 18:16 07-May-20
  
```

Figure 6.17: Decentralised fault tolerance — Round 46. All surviving nodes continue making progress, with Nodes 2 and 4 showing reduced accuracy due to limited blending after Node 3's exit.

6.5.3 The Cascading Timing-Drift Effect

An unexpected finding emerged from the fault experiment. Because the synchronous gossip protocol requires each node to wait for both neighbours to push their models, Node 3’s absence caused Nodes 2 and 4 to wait for the full 90-second push retry timeout before proceeding. This inflated their round duration from approximately 37 seconds to approximately 130 seconds per round.

This timing inflation had a cascading effect. When Nodes 2 and 4 consistently ran 90 seconds late per round, their surviving neighbours — Node 1 (adjacent to Node 2) and Node 5 (adjacent to Node 4) — began timing out on their push attempts to Nodes 2 and 4 as well. By Round 38, Node 2 had lost connectivity to both its remaining neighbours and began training in complete isolation. Node 4 followed at Round 41.

The timing disruption propagated outward from the failed node with measurable distance decay:

Table 6.5: Communication time inflation by ring distance from failed Node 3

Distance from Node 3	Avg comm time	Inflation
1 hop (Nodes 2, 4)	~96–130s	+303%
2 hops (Nodes 1, 5)	~65s	+172%
3 hops (Nodes 0, 6)	~51s	+114%
4 hops (Node 7)	~49s	+104%

The accuracy impact was localised to the two direct neighbours. Nodes 2 and 4 finished with final accuracies of 43.70% and 40.72% respectively — drops of 7.63 pp and 9.13 pp compared to their baselines in Experiment 4. All other 5 surviving nodes finished within ± 2 pp of their normal Non-IID baselines.

This cascading isolation behaviour — where a single failed node eventually causes its direct neighbours to lose *all* connectivity through secondary timing effects — was not predicted by theoretical gossip FL models. It is a real-world TCP timeout behaviour that only becomes visible on actual distributed infrastructure.

6.5.4 Direct Comparison of Fault Outcomes

Table 6.6 summarises the direct comparison between Experiments 5-A and 5-B.

Table 6.6: Fault experiment outcomes: decentralised vs centralised

Metric	Exp 5-A (Decentralised)	Exp 5-B (Centralised)
Who fails	Node 3 (1 of 8)	Server (sole coordinator)
Nodes that continue	7/8 — 50/50 rounds	0/7 — stuck at R9
System halt?	No	Yes, permanent
Final accuracy	48.68% avg (7 nodes)	49.07% (frozen)
Accuracy lost	7.6–9.1 pp (2 nodes)	~8.5 pp (all 7 nodes)
Detection time	~90s	~4 min 14s (irrelevant)
Recovery possible?	Yes — partial	No

The comparison is stark. In centralised FL, 1 server failure equals 7 client failures and 100% system halt. In decentralised ring gossip, 1 node failure equals 2 degraded neighbours and 5 unaffected nodes, with 0% system halt. The 2 directly affected nodes lost accuracy but never stopped training. In centralised FL, all 7 clients lost 82% of their planned training budget instantly and permanently.

Chapter 7

Conclusion

7.1 Summary of Work

This thesis set out to answer a question that the federated learning literature had left unanswered: what actually happens when a server or node fails during training on real distributed infrastructure, and how much accuracy does a decentralised architecture sacrifice to eliminate that risk?

To answer this, both a centralised FedAvg system and a decentralised synchronous ring gossip FL system were built from scratch in Python and PyTorch, deployed on 8 real AWS EC2 instances, and evaluated across six controlled experiments covering IID and Non-IID data distributions, normal operation, and live failure injection. An earlier two-node pilot on VMware VMs confirmed the P2P communication infrastructure and produced the unplanned server crash at Round 17 that directly motivated the full fault injection experiments.

7.2 Key Findings

The accuracy cost of decentralisation is small and consistent. Decentralised gossip FL reached within 4.21 pp of centralised FedAvg under IID conditions (55.98% vs 60.02% final accuracy) and within 7.09 pp under Non-IID (49.49% vs 56.58%). Both architectures converged to 50% accuracy within a similar number of rounds under IID. The accuracy gap is real but modest, and represents the measurable price of eliminating the central server entirely.

Gossip FL is more sensitive to Non-IID data than FedAvg. The IID-to-Non-IID accuracy drop was 6.49 pp in the decentralised setting versus 3.44 pp in the centralised setting. This is because each gossip node blends with only 2 immediate neighbours per round, making it vulnerable to correlated class imbalances in its local neighbourhood. FedAvg aggregates all 7 clients every round and sees the full diversity of class distributions, making it more robust to data heterogeneity.

The SPOF is real and total. When the server was killed after Round 9, all 7

clients halted permanently within 4 minutes and 14 seconds. After 80 upload attempts each, every client logged `CRITICAL --- SERVER UNREACHABLE`. The system froze at 49.07% accuracy with 41 rounds of training permanently lost. No client could continue or recover without a manual server restart.

Decentralised FL survives node failure. When Node 3 was killed after Round 10, all 7 remaining nodes completed all 50 rounds. The system never halted. Direct neighbours Node 2 and Node 4 saw accuracy drops of 7.63 pp and 9.13 pp respectively, but continued training throughout. All other 5 nodes finished within ± 2 pp of their normal baselines.

A previously undescribed cascading failure behaviour was discovered. When Node 3 failed, the 90-second TCP push retry timeout at its direct neighbours inflated their round duration from ~ 37 seconds to ~ 130 seconds. This timing drift cascaded outward: Nodes 2 and 4 eventually lost synchronisation with their surviving neighbours as well, becoming fully isolated by Rounds 38 and 41 respectively. The timing inflation decayed measurably with ring distance but was still detectable 4 hops away. This emergent behaviour — where a single node failure causes secondary neighbour isolation through TCP timeout accumulation — is not predicted by any existing theoretical gossip FL model.

7.3 Limitations

Scale. All experiments used 8 nodes. The behaviour of ring gossip at larger scales — 16, 32, or 64 nodes — was not studied. Knowledge propagation time grows with ring size, and the cascading timing-drift effect may behave differently in larger rings.

Synchronous protocol only. Both architectures used a synchronous protocol where every node waits for its partners before proceeding. Asynchronous FL — where nodes train and exchange independently — is a different research track with its own convergence and fault tolerance properties. This thesis does not address it.

Fixed ring topology. Each node always communicated with the same two neighbours. Dynamic gossip topologies, where neighbourhood changes each round, may propagate knowledge faster and be more resilient to failures. This was explicitly out of scope.

Single dataset and model. All experiments used CIFAR-10 with a simple CNN. Whether the findings generalise to larger models, different datasets, or more complex architectures is an open question.

No Byzantine fault tolerance. The experiments tested clean node exits, not malicious or corrupted nodes. Byzantine fault tolerance in ring gossip is a separate and significant research problem.

7.4 Future Work

Several directions follow naturally from this work.

Adaptive timeout backoff. The cascading timing-drift effect was caused by a fixed 90-second push retry timeout. A simple improvement would be to detect consistent push failures after 2–3 attempts and reduce the wait time accordingly, preventing secondary isolation of direct neighbours.

Scaling experiments. Running the same controlled comparison at 16, 32, and 64 nodes would characterise how the accuracy gap and fault tolerance behaviour scale with ring size. This is the most direct extension of this work.

Asynchronous gossip FL. Porting the ring gossip system to an asynchronous protocol would allow comparison of synchronous and asynchronous decentralised FL under the same fault injection conditions.

Dynamic topology. Replacing the fixed ring with random or adaptive gossip — where each node selects different neighbours each round — would test whether faster knowledge propagation reduces the Non-IID sensitivity gap.

Cross-network deployment. Extending the experiments beyond a single AWS VPC to nodes in different regions or different cloud providers would introduce real network variability and test the architecture under conditions closer to actual federated deployments.

REFERENCES

- [1] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (pp. 1273–1282).
- [2] Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. *Proceedings of Machine Learning and Systems*, 2, 429–450.
- [3] Bonawitz, K., Eichner, H., Grieskamp, W., Huba, D., Ingerman, A., Ivanov, V., Kiddon, C., Konečný, J., Mazzocchi, S., McMahan, H. B., Van Overveldt, T., Petrou, D., Ramage, D., & Roselander, J. (2019). Towards federated learning at scale: System design. *Proceedings of Machine Learning and Systems*, 1, 374–388.
- [4] Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., et al. (2021). Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2), 1–210.
- [5] Lian, X., Zhang, C., Zhang, H., Hsieh, C.-J., Zhang, W., & Liu, J. (2017). Can decentralized algorithms outperform centralized algorithms? A case for decentralized stochastic gradient descent. In *Advances in Neural Information Processing Systems* (NeurIPS), 30.
- [6] Lalitha, A., Shekhar, S., Javidi, T., & Koushanfar, F. (2019). Fully decentralized federated learning. In *NeurIPS Workshop on Bayesian Deep Learning*.
- [7] Boyd, S., Ghosh, A., Prabhakar, B., & Shah, D. (2006). Randomized gossip algorithms. *IEEE Transactions on Information Theory*, 52(6), 2508–2530.
- [8] Beltran, E. T. M., et al. (2023). Decentralized federated learning: Fundamentals, state-of-the-art, frameworks, trends, and challenges. *IEEE Communications Surveys & Tutorials*, 25(4), 2983–3013.

-
- [9] Roy, A. G., Siddiqui, S., Pölsterl, S., Navab, N., & Wachinger, C. (2019). Brain-Torrent: A peer-to-peer environment for decentralized federated learning. *arXiv preprint arXiv:1905.06731*.
 - [10] Xu, J., & Wang, F. (2021). Federated learning for healthcare informatics. *Journal of Healthcare Informatics Research*, 5(1), 1–19.
 - [11] Li, Q., Wen, Z., Wu, Z., Hu, S., Wang, N., Li, Y., Liu, X., & He, B. (2021). A survey on federated learning systems: Vision, hype and reality for data privacy and protection. *IEEE Transactions on Knowledge and Data Engineering*.
 - [12] Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3), 382–401.
 - [13] Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. *ACM SIGOPS Operating Systems Review*, 33(5), 173–186.